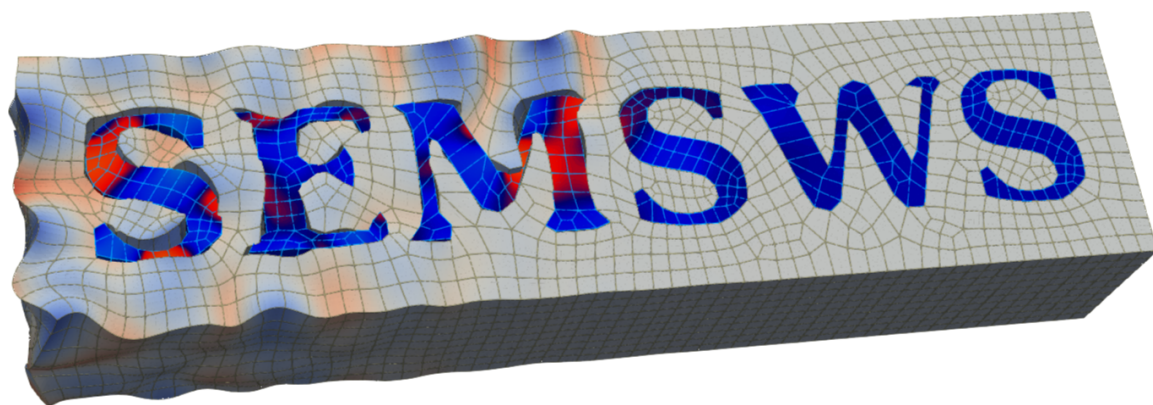




Spectral Element Method
Seismic Wave Simulator

ユーザーマニュアル

Version 0.1

2026 年 5 月 12 日

目次

第 1 章	はじめに	5
1.1	SEMSWS とは	5
1.2	対応する物理モデル	5
1.3	スペクトル要素法の要点	6
1.4	本マニュアルの構成	6
第 2 章	インストール	7
2.1	動作要件	7
2.2	依存ライブラリのビルド	8
2.3	SEMSWS のビルド	9
2.4	ビルド設定例	10
2.5	Spack によるインストール (LUMI / Earth Simulator)	13
2.6	ビルド結果の確認	22
2.7	GLVis のインストール (推奨)	23
第 3 章	クイックスタート	25
3.1	設定ファイル	25
3.2	シミュレーションの実行	28
3.3	結果の確認	28
第 4 章	入力ファイルリファレンス	31
4.1	YAML 形式の基礎	31
4.2	厳密キー検証 (Strict-Key Validation)	32
4.3	設定ファイルの全体構造	32
4.4	ルートレベルパラメータ	33
4.5	simulation セクション	33
4.6	mesh セクション	37
4.7	material セクション	39
4.8	boundary セクション	45
4.9	sources セクション	47
4.10	receivers セクション	51
4.11	device セクション	54

4.12	inversion セクション	55
4.13	HDF5 入出力スキーマと使い方	55
第 5 章	ツールとスクリプト	67
5.1	partition_mesh — メッシュの事前分割	67
5.2	combine_mesh — 分割メッシュの結合	68
5.3	evaluate_mesh — メッシュ品質の評価	69
5.4	refine_mesh — メッシュの均一細分化と Partitioning	69
5.5	ワークフロー例：メッシュ評価と細分化	71
5.6	可視化スクリプト	74
第 6 章	応用例	79
6.1	visualization — 可視化デモ	79
6.2	elastic_analytic — 解析解ベンチマーク	79
6.3	visco_elastic_3d_layered — 3D 粘弾性 2 層モデル	80
6.4	fun	81
第 7 章	パフォーマンス	85
7.1	テスト環境	85
7.2	Strong Scaling	85
7.3	Weak Scaling	86
7.4	GPU vs CPU	88
開発チーム		89

第 1 章

はじめに

1.1 SEMSWS とは

SEMSWS (Spectral Element Method – Seismic Wave Simulator) は、スペクトル要素法 (SEM) に基づく地震波伝播シミュレーションソルバである。有限要素法ライブラリ MFEM^{*1}上に構築されている。MFEM は Lawrence Livermore National Laboratory を中心に開発されているオープンソースライブラリであり、高次有限要素空間、多様なメッシュフォーマットのサポート、MPI 並列アセンブリ、CUDA/HIP/OpenMP バックエンドによる GPU/CPU ポータビリティなど、SEMSWS の中核機能の多くは MFEM の基盤に支えられている。MFEM の開発チームに深く感謝する。これにより、単一ソースコードで CPU および GPU (CUDA/HIP) 上での実行に対応する。

地震波動方程式の数値解法として広く用いられる SEM は、Gauss–Lobatto–Legendre (GLL) 点に基づく高次 Lagrange 補間を用いる手法であり、質量行列の対角化による効率的な陽的時間積分、非構造メッシュへの対応、弱形式による自由表面境界条件の自然な取り込み、そしてスペクトル収束に基づく高精度かつ低分散な解が得られるという特長を持つ。

1.2 対応する物理モデル

SEMSWS は以下の波動方程式を 2 次元・3 次元でサポートする：

- 等方性弾性波動方程式 (Isotropic Elastic)
- 等方性音響波動方程式 (Isotropic Acoustic)
- 粘弾性・粘音響減衰 (Viscoelastic / Viscoacoustic)

今後のアップデートで VTI、Full Anisotropy、固体–液体混合領域などのモデルへの対応も予定している。

多項式次数は任意に設定可能であるが、GPU 実装では剛性カーネルの共有メモリ制約により上限が存在する。

時間積分には陽的 Newmark スキーム ($\beta = 0$, $\gamma = 1/2$ 、中心差分法と等価) を用いる。このスキームは時間 2 次精度であり、CFL 条件のもとで条件付き安定である。

吸収境界条件としては、スポンジ吸収層 (Cerjan damping) を実装している。完全整合層 (PML)

^{*1} <https://mfem.org/>

は将来の開発で対応予定である。

Full Waveform Inversion (FWI) 機能は現在開発中である、Acoustic 2D でのみ一部対応しているが、テストは不十分な状態である。

1.3 スペクトル要素法の要点

ユーザーが設定ファイルで指定する主な SEM パラメータは**多項式次数 (order)** である。

$\text{order} = N$ のとき、各要素の各辺に沿って $N + 1$ 個の GLL 点が配置される。地震波シミュレーションでは $\text{order} = 4 \sim 5$ が一般的である。

SEM の本質的な利点は、GLL 点を補間点と積分点の両方に用いることで質量行列が対角化される点にある。これにより、加速度場の計算が要素ごとの単純な除算で済み、連立方程式の求解が不要となる。剛性項 $\mathbf{KU}$ は行列として組み立てず、要素レベルの演算により matrix-free に計算されるため、メモリ使用量と計算コストの両方が抑えられる。

1.4 本マニュアルの構成

第2章：インストール 依存ライブラリ (hypr, MFEM) の準備からビルドまでの手順を説明する。

各計算環境 (ローカル、ES、LUMI) でのビルド設定例も掲載。

第3章：クイックスタート 2D 音響波シミュレーションを例に、設定ファイルの記述から実行、結果の可視化までの一連の流れを説明する。

第4章：入力ファイルリファレンス config.yaml の全オプションを網羅的に記述する。

第5章：ツールとスクリプト メッシュの分割・細分化・品質評価ツールと可視化スクリプトの使い方説明する。

第6章：応用例 解析解ベンチマーク、3D 粘弾性モデルなどの例題を紹介する。

第7章：パフォーマンス ES・LUMI でのスケーリング結果と GPU/CPU 性能比較を示す。

第 2 章

インストール

2.1 動作要件

SEMSWS のビルドには以下のソフトウェアが必要である。

2.1.1 必須

表 2.1 必須の依存ライブラリ

ソフトウェア	備考
C++17 コンパイラ	-std=c++17 サポート
CMake	ビルドシステム
MPI	並列計算
MFEM	MPI 有効でビルドすること
ADIOS2	並列 I/O (BP 形式)
HDF5	波形出力
yaml-cpp	CMake が自動取得 (手動不要)

2.1.2 オプション

表 2.2 オプションの依存ライブラリ

ソフトウェア	備考
CUDA	NVIDIA GPU サポート
HIP	AMD GPU サポート

2.1.3 pytest を実行する場合の Python パッケージ

test/以下のテストスイートを実行する場合は、Python 3.10 以上に加えて以下のパッケージを pip でインストールする：

表 2.3 pytest に必要な Python パッケージ

パッケージ	必須/任意	用途
pytest	必須	テストランナー本体
numpy	必須	波形比較・数値検証
h5py	必須	HDF5 入出力テスト (test/consistency/など)
pyyaml	必須	YAML 設定ファイルの読み込み
matplotlib	任意	プロット系テストおよび診断画像
gmsh	任意	流体-固体連成のメッシュ生成テスト (test/coupling/)

一括インストール例：

```
pip install pytest numpy h5py pyyaml matplotlib gmsh
```

driver/以下の Python ドライバ (semsws-driver) も独自の依存をもつ。そちらは `pip install -e ./driver` で driver/pyproject.toml から自動解決される。

2.2 依存ライブラリのビルド

SEMSWS は MFEM ライブラリ上に構築されており、MFEM はさらに hypre に依存する。これらは精度設定や GPU バックエンドを揃えてビルドする必要がある。

2.2.1 hypre のビルド

■CPU 版

```
cd hypre/src
./configure --enable-single --with-MPI
make -j$(nproc)
```

倍精度の場合は `--enable-single` を省略する。

■CUDA 版 (NVIDIA GPU)

```
cd hypre/src
./configure \
  --enable-single --enable-mixedint \
  --enable-unified-memory \
  --with-cuda --enable-gpu-aware-mpi \
  CUDA_HOME=/path/to/cuda HYPRE_CUDA_SM=80
make -j$(nproc)
```

■HIP 版 (AMD GPU)

```
cd hypre/src
./configure \
  --enable-single --enable-mixedint \
  --enable-unified-memory \
  --with-hip --enable-gpu-aware-mpi \
  CC=hipcc CXX=hipcc
make -j$(nproc)
```


-enable-unified-memory は、おそらく GPU 環境で推奨。

2.2.2 MFEM のビルド

MFEM は SEMSWS の中核となる有限要素ライブラリである。CMake でビルドし、以下のオプションを設定する。

■ビルド例 (CPU 版・単精度)

```
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_CXX_COMPILER=mpicxx \
  -DMFEM_USE_MPI=ON \
  -DMFEM_USE_HDF5=ON \
  -DHDF5_DIR=/path/to/hdf5 \
  -DMFEM_USE_NETCDF=ON \
  -DMFEM_PRECISION=single \
  -DHYPRE_DIR=/path/to/hypre/src/hypre
cmake --build build -j$(nproc)
```

NETCDF は必須でないが、CUBITなどで生成したメッシュの読み込みに必要となる。

■精度の一貫性 hypre の精度設定 (--enable-single) と MFEM の精度設定 (MFEM_PRECISION) は一致させる必要がある。SEMSWS は MFEM の精度設定を自動的に引き継ぐため、hypre と MFEM を揃えておけば SEMSWS 側での設定は不要である。

MFEM のビルドの詳細については、公式ドキュメント^{*1}を参照されたい。

2.3 SEMSWS のビルド

2.3.1 基本的なビルド手順

ソースコードのルートディレクトリから以下のコマンドを実行する：

```
# 1. CMake configure
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_INSTALL_PREFIX=<install_dir> \
  -DMFEM_DIR=/path/to/mfem/build

# 2. Build
cmake --build build -j$(nproc)

# 3. Install (実行ファイルを CMAKE_INSTALL_PREFIX に配置)
cmake --install build
```

MFEM_DIR には MFEM のビルドディレクトリまたはインストールディレクトリパスを指定する。CMAKE_INSTALL_PREFIX を指定して cmake --install build を実行すると、<prefix>/bin/

^{*1} <https://mfem.org/building/>

に `semsws` や周辺ツールが、`<prefix>/lib/` に静的ライブラリが、`<prefix>/include/semsws/` にヘッダがそれぞれ配置される。

2.3.2 CMake オプション

表 2.4 CMake オプション一覧

オプション	デフォルト	説明
<code>MFEM_DIR</code>	—	MFEM のパス (必須)
<code>CMAKE_BUILD_TYPE</code>	—	Release/Debug
<code>CMAKE_INSTALL_PREFIX</code>	<code>/usr/local</code>	<code>cmake --install build</code> での配置先
<code>CMAKE_CXX_COMPILER</code>	自動検出	MPI コンパイラ
<code>CMAKE_CUDA_ARCHITECTURES</code>	<code>70;80</code>	CUDA アーキテクチャ
<code>AMDGPU_TARGETS</code>	<code>gfx90a</code>	AMD GPU アーキテクチャ
<code>SEM_USE_GPU_AWARE_MPI</code>	OFF	リンクする MPI が GPU-aware (CUDA/ROCm 対応) であることをビルド時に表明する。OFF (既定) では流体-固体結合のバッファをホスト経由でステージングする (任意の MPI で安全)。ON は、LUMI Cray MPICH (<code>MPICH_GPU_SUPPORT_ENABLED=1</code>) や CUDA 対応 OpenMPI など、GPU-aware が確認されている環境でのみ指定すること (誤った ON 指定は MPI_Isend 内でクラッシュする)。

2.3.3 GPU ビルド

MFEM が CUDA または HIP 付きでビルドされている場合、SEMSWS は自動的に GPU ビルドとなる。

2.4 ビルド設定例

以下に、各環境でのビルド手順を示す。

2.4.1 ローカル環境 (WSL2/Ubuntu, CPU only)

単精度・CPU のみの構成例。

```
# hypr
cd $HOME/program_ubuntu/hypr/src
./configure --enable-single --with-MPI
make -j$(nproc)

# MFEM
```

```

cd $HOME/program_ubuntu/mfem
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_CXX_COMPILER=mpicxx \
  -DMFEM_USE_MPI=ON \
  -DMFEM_USE_HDF5=ON -DMFEM_USE_NETCDF=ON \
  -DMFEM_PRECISION=single \
  -DHYPRE_DIR=$HOME/program_ubuntu/hypre/src/hypre
cmake --build build -j$(nproc)

# SEMSW
cd $HOME/program_ubuntu/SEMSW
cmake -B build \
  -DCMAKE_BUILD_TYPE=Debug \
  -DCMAKE_INSTALL_PREFIX=<install_dir> \
  -DMFEM_DIR=$HOME/program_ubuntu/mfem/build
cmake --build build -j$(nproc)
cmake --install build

```

2.4.2 Earth Simulator (NVIDIA A100 GPU)

JAMSTEC の Earth Simulator の GPU パーティション (NVIDIA A100) での例。コンパイラには NVIDIA HPC SDK 24.7 (nvc++/nvcc) を使用する。2025 年 4 月時点で動作確認済みの構成は以下の通り：NVIDIA HPC SDK 24.7、CUDA 12.0.0、hypre 2.31.0、METIS 5.1.0、HDF5 1.10.5。

```

# モジュールのロード
module load HDF5/1.10.5
module load NetCDF4/all
module load NVIDIAHPCSDK/24.7/nvhpc-hpcx-cuda12
module load METIS/5.1.0
module load CUDA/12.0.0

# hypre
cd $HOME/program_gpu/hypre/src
./configure \
  --enable-single --enable-mixedint \
  --enable-unified-memory \
  --with-cuda --enable-gpu-aware-mpi \
  CUDA_HOME=/opt/share/CUDA/12.0.0 HYPRE_CUDA_SM=80
make -j$(nproc)

# MFEM (上でビルドした hypre を使う)
cd $HOME/program_gpu/mfem
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_CXX_COMPILER=nvc++ \
  -DCMAKE_CUDA_COMPILER=nvcc \

```

```

-DCMAKE_CUDA_ARCHITECTURES=80 \
-DMFEM_USE_MPI=ON \
-DMFEM_USE_CUDA=ON \
-DMFEM_USE_HDF5=ON \
-DMFEM_USE_NETCDF=ON \
-DMFEM_PRECISION=single \
-DHYPRE_DIR=$HOME/program_gpu/hypre/src/hypre \
-DMETIS_DIR=/opt/share/METIS/5.1.0
cmake --build build -j$(nproc)

# SEMSW
cd $HOME/program_gpu/SEMSW
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_INSTALL_PREFIX=<install_dir> \
  -DCMAKE_CXX_COMPILER=nvc++ \
  -DCMAKE_CUDA_COMPILER=nvcc \
  -DCMAKE_CUDA_ARCHITECTURES=80 \
  -DCMAKE_CXX_STANDARD=17 \
  -DSEM_USE_GPU_AWARE_MPI=ON \
  -DMFEM_DIR=$HOME/program_gpu/mfem/build \
  -Dadios2_DIR=$HOME/program_cpu/ADIOS2/build
cmake --build build -j
cmake --install build

```

SEMSW の cmake が受け取るのは MFEM (MFEM_DIR) と ADIOS2 (adios2_DIR) のみ。hypre や METIS は MFEM 経由で推移的にリンクされるため、SEMSW 側の cmake 行には現れない。

2.4.3 LUMI (AMD MI250X GPU)

CSC Finland の LUMI スーパーコンピュータ (LUMI-G パーティション) での例。コンパイラには ROCm 6.4.4 の hipcc、MPI には Cray MPICH を用いる。2026 年 4 月時点で動作確認済みの構成は以下の通り：ROCm 6.4.4、Cray PE 25.09 (CCE 20.0.0)、hypre 3.1.0、METIS 5.1.0 (cpeCray-25.09-idx32-fp32)、cray-hdf5 1.14.3.7。

```

# モジュールのロード
module load LUMI/25.09 partition/G
module load METIS/5.1.0-cpeCray-25.09-idx32-fp32
module load cray-hdf5/1.14.3.7

# hypre
cd $FD/program/hypre/src
./configure \
  --enable-single --enable-mixedint \
  --enable-unified-memory \
  --with-hip --enable-gpu-aware-mpi \
  CC=hipcc CXX=hipcc \
  LDFLAGS="-L/opt/cray/pe/mpich/9.0.1/gtl/lib" \

```

```
LIBS="-lmpi_gtl_hsa"
make -j$(nproc)

# MFEM (上でビルドしたhypreを使う)
cd $FD/program/mfem
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_CXX_COMPILER=hipcc \
  -DMFEM_USE_MPI=ON \
  -DMFEM_USE_HIP=ON \
  -DAMDGPU_TARGETS=gfx90a \
  -DMFEM_USE_HDF5=ON \
  -DMFEM_USE_NETCDF=ON \
  -DMFEM_PRECISION=single \
  -DHYPRE_DIR=$FD/program/hypre/src/hypre \
  -DHDF5_INCLUDE_DIRS=/opt/cray/pe/hdf5/1.14.3.7/cray/20.0/include \
  -DHDF5_LIBRARIES="/opt/cray/pe/hdf5/1.14.3.7/cray/20.0/lib/libhdf5_cpp.so
    ;\
/opt/cray/pe/hdf5/1.14.3.7/cray/20.0/lib/libhdf5.so"
cmake --build build -j$(nproc)

# SEMSWs
cd $FD/program/SEMSWS
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_INSTALL_PREFIX=<install_dir> \
  -DCMAKE_CXX_COMPILER=hipcc \
  -DAMDGPU_TARGETS=gfx90a \
  -DSEM_USE_GPU_AWARE_MPI=ON \
  -DMFEM_DIR=$FD/program/mfem/build \
  -DAdios2_DIR=$FD/program/ADIOS2/build
cmake --build build -j
cmake --install build
```

SEMSWS の cmake が受け取るのは MFEM (MFEM_DIR) と ADIOS2 (adios2_DIR) のみ。hypre・METIS・HDF5 の位置は MFEM 経由で推移的にリンクされるため、SEMSWS 側の cmake 行には現れない。MPI ジョブは `srun` で投入する。MPI ジョブの実行には `srun` を使用する。

2.5 Spack によるインストール (LUMI / Earth Simulator)

前節までの手動ビルドは依存ライブラリのバージョン整合性を人手で管理する必要があり、サイト間移植が煩雑である。SEMSWS リポジトリには `spack-repo/`以下に Spack レシピと `site-config/lumi/`・`site-config/es/`の事前設定一式が同梱されており、Spack 経由で依存関係を含む全ビルドを自動化できる。

本節では LUMI と Earth Simulator (以下 ES) での Spack 利用手順を、各コマンドと各フラグがなぜ必要かを明示しながら示す。

2.5.1 Spack 版の長所と用途

- **依存の自動解決**：hydre / MFEM / ADIOS2 / HDF5 / NetCDF / METIS / openblas のバージョン制約を SEMSWS の `package.py` に集約済み。
- **サイト固有の罣を回避**：LUMI では cray-mpich GTL lib のリンク、ES では NVHPC の prefix 展開バグ・nvc 特有の autotools 不整合・cuSOLVER の単精度 FSAI バグ・hydre 3.x の CMake 破綻などをすべて SEMSWS の shadow recipe で吸収済み。
- **再現性**：spack env で固定した spack.yaml + spack.lock によってビルド構成が完全再現できる。

■Spack のバージョン要件. SEMSWS の `package.py` は Spack v1.0 で導入された明示的言語宣言 (`depends_on("c", type="build")・depends_on("cxx", type="build")`) を使用するため、Spack 1.0 以上が必須である。動作確認済みは v1.1.x (stable) と v1.2.x (develop branch)。

2.5.2 最も簡単な例（汎用 Linux 環境）

LUMI/ES 固有の調整が不要な手元の Linux マシンや汎用クラスタでは、3 コマンドでビルド・インストールできる。

```
# 1. SEMSWS のリポジトリを取得
git clone https://github.com/SEMSWS/semsws.git
cd SEMSWS

# 2. SEMSWS の Spack repo を登録（1回だけ）
spack repo add ./spack-repo

# 3. インストール（依存パッケージを含めて自動ビルド）
spack install semsws@0.1.0
```

semsws@0.1.0 は tag 付き release を指す。最新の開発版を入れる場合は semsws@main を指定する。

■推奨：CMake を external に登録. SEMSWS は cmake@3.20: を要求する。Spack に任せると cmake もソースからビルドされて 10 分以上かかるため、システム cmake (バージョン 3.20 以上) を external として登録するのが推奨。~/.spack/packages.yaml に以下を記述：

```
packages:
  cmake:
    externals:
      - spec: cmake@3.28.0          # `cmake --version` の出力に合わせる
        prefix: /usr               # `which cmake` で確認した prefix
        buildable: false          # 必ずこの external を使う
```

同様に git や python など既にシステムに入っているビルドツールは external にすると初回ビルド時間が大幅に短縮される。

■使い方

```
spack load semsws
which semsws      # /path/to/spack/opt/.../semsws-0.1.0-.../bin/semsws
mpirun -np 4 semsws --config config.yaml
```

LUMI/ES のようにサイト固有の external 宣言や精度・GPU・MPI ラッパーの調整が必要な環境では、以下の専用節を参照する。

2.5.3 LUMI (AMD MI250X GPU) での Spack ビルド

LUMI partition/G、ROCm 6.4.4、cray-mpich 9.0.1、CCE 20.0.0、hypre 3.1.0、cray-hdf5 1.14.3.7 構成。

■初回セットアップ

```
# 0) project 番号 (自分の allocation に合わせる)
export PROJECT=project_465002833

# 1) Spack を flash へ clone (プロジェクト共有可、1回だけ)
git clone --depth=1 https://github.com/spack/spack.git \
  /flash/${PROJECT}/program/spack
```

- --depth=1: 履歴を 1 コミットに限定。Spack 本体の履歴は不要。
- /flash/...: LUMI のスクラッチ層。一般 home (/users/) には inode が足りない (Spack は数万ファイル生成する) ため flash 必須。

```
# 2) SEMSWs を clone
cd /flash/${PROJECT}/${USER}
git clone https://github.com/SEMSWS/semsws.git
cd SEMSWs

# 3) site config をコピー
cp spack-repo/site-config/lumi/load.sh ~/load.sh
mkdir -p ~/.spack
cp spack-repo/site-config/lumi/packages.yaml ~/.spack/packages.yaml
```

- cp packages.yaml ~/.spack/: Spack の user scope。externals として登録する cray PE パッケージ群 (cce、cray-mpich、cray-hdf5、cray-libsci、ROCm 6.4.4 各コンポーネント) の prefix が書かれており、Spack がこれらを再ビルドせず既存 module をそのまま使う。

■環境ロード

```
source ~/load.sh
spack compiler find
```

load.sh が行うこと:

- module load LUMI/25.09 partition/G: LUMI-G サイト設定。
- module load PrgEnv-cray: CCE コンパイラ環境。

- `module load cray-mpich/9.0.1 craype-accel-amd-gfx90a:GPU-aware` Cray MPICH。後者がないと MPICH GTL lib が正しく注入されない。
- `module load rocm/6.4.4 lumi-CrayPath:ROCm` 本体と Cray PATH 整合パッチ。
- `export MPICH_GPU_SUPPORT_ENABLED=1` : cray-mpich に GPU バッファ直接送信を許可。これを忘れると SEMSWs の `+gpu_aware_mpi` 指定が実行時に `MPI_Isend` 内部で落ちる。
- `export SPACK_USER_CACHE_PATH=/flash/...` : build stage とダウンロードキャッシュを flash へ。home に置くと inode 枯渇。
- `source $SPACK_ROOT/share/spack/setup-env.sh` : Spack のシェル関数を有効化。

`spack compiler find` は LUMI モジュール経由で見える CCE 20.0.0 を `~/.spack/compilers.yaml` に登録する。

■インストール

```
# SEMSWs の spack repo を登録
spack repo add /flash/${PROJECT}/${USER}/SEMSWS/spack-repo

# 本番 (GPU + 単精度 + GPU-aware MPI)
spack install semsws@main +rocm amdgpu_target=gfx90a \
    precision=single +gpu_aware_mpi
```

各フラグの意味：

- `@main` : SEMSWs リポジトリの `main` ブランチに対応する version entry。tag 付き release を使う場合は `semsws@0.1.0` のように指定する。
- `+rocm` : HIP backend 有効化。下流の `mfem`・`hypre` にも自動伝播。
- `amdgpu_target=gfx90a` : MI250X (CDNA2) の ISA ターゲット。`hypre`・`mfem`・SEMSWS 全部に伝播し、CMake の `HIP_ARCHITECTURES` や `hypre` の `--with-amdgpu-arch` に展開される。
- `precision=single` : `mfem`・`hypre` にも伝播し全体を単精度化。FWI ではメモリ使用量が半分、A100/MI250X 上で 2 倍の演算スループット。
- `+gpu_aware_mpi` : `SEM_USE_GPU_AWARE_MPI` 定義に加え、`hypre` に `+gpu-aware-mpi` が伝播、さらに SEMSWs の CMake `flag_handler` が `cray-mpich` の GTL ライブラリ (`libmpi_gtl_hsa`) を実行ファイルに明示リンクする。これがないと実行時に `MPI_Isend` 内で crash する。

■ロードと使用

```
spack load semsws
which semsws
# /flash/.../spack/opt/spack/.../semsws-main-.../bin/semsws

# srun 経由で MPI ジョブ実行
srun -A ${PROJECT} -p standard-g --nodes=1 --ntasks=8 --gpus-per-node=8 \
    semsws config.yaml
```


2.5.4 Earth Simulator (NVIDIA A100 GPU) での Spack ビルド

ES4 の GPU パーティション、NVHPC 24.7 + HPC-X OpenMPI + CUDA 12.0.0 構成。LUMI と違い NVHPC の分割 CUDA 配置 (cuda/と math_libs/が別ディレクトリ) や nvc の -m32 非対応など固有の罣が複数あり、SEMSWS 側で shadow レシピを多数用意して対処している。

ES では GPU ビルドと CPU ビルドの 2 系統を独立に管理できるよう、site-config/es/にロードスクリプトと packages.yaml を各々 load_gpu.sh・load_cpu.sh・packages_gpu.yaml・packages_cpu.yaml として分けて配置している。

■初回セットアップ (GPU・CPU 共通)

```
export WORK=/S/data01/G3506/${USER}/program_test
mkdir -p $WORK

# Spackをworkへclone
git clone --depth=1 https://github.com/spack/spack.git $WORK/spack

# SEMSWs
cd $WORK
git clone https://github.com/SEMSWS/semsws.git
cd SEMSWs

# ロードスクリプトとpackages.yamlをコピー
cp spack-repo/site-config/es/load_gpu.sh ~/load_gpu.sh
cp spack-repo/site-config/es/load_cpu.sh ~/load_cpu.sh
chmod +x ~/load_*.sh
```

■GPU ビルド環境のロード

```
source ~/load_gpu.sh
```

load_gpu.sh が行うこと：

- module load NVIDIAHPCSDK/24.7/nvhpc-hpcx-cuda12:NVHPC SDK 24.7 をロード。同時に HPC-X OpenMPI (CUDA 12.4 ビルド) も入る。
- module load CUDA/12.0.0:standalone CUDA 12.0 モジュール。NVHPC 同梱の 12.5 は cuda/と math_libs/が分割配置で MFEM の cusparse.h 検出に失敗するため、flat layout の 12.0 を別途使う (manual ビルドの semsws_manual ch02 ES と同構成)。
- module load METIS/5.1.0・HDF5/1.10.5・NetCDF4/all:システムモジュール。ただし Spack ビルドではこれらの大半を **Spack に再ビルドさせる** (理由は後述)。
- module load Python/3.12.6:pytest 等で使う。
- export PATH=/usr/bin:\$PATH:ES のバッチサンドボックスで /usr/bin/git が見えなくなるので明示的に前置。
- export SPACK_GIT_COMMAND=/usr/bin/git・GIT_PYTHON_GIT_EXECUTABLE=/usr/bin/git: Spack 1.x 起動時の「spack requires 'git'」チェックが sanitized PATH で git を探すので、明示的に教える。
- export SPACK_USER_CONFIG_PATH=\$WORK/.spack:home でなく work を使うことで複数

env 構成を 1 か所に集約。

■Spack env (GPU) の作成 GPU と CPU を並立させるため Spack environment を用いる。

```
spack compiler find /usr/bin
# nvhpc@24.7 + gcc@8.5.0 が登録される

spack repo add $WORK/SEMSWS/spack-repo
spack env create semsws-gpu
spack env activate semsws-gpu
spack config edit
```

エディタが開いたら、spack-repo/site-config/es/packages_gpu.yaml の内容を spack:packages: 節以下にネストして貼り付ける。完成形は：

```
spack:
  repos:
    - /S/data01/G3506/${USER}/program_test/SEMSWS/spack-repo
  packages:
    # ... packages_gpu.yaml の中身をここに ...
  specs:
    - semsws@main +cuda cuda_arch=80 precision=single +gpu_aware_mpi
      cxxflags=="-noswitcherror" cflags=="-noswitcherror"
      ldlibs=="-lstdc++fs"
      ^hypre@2.33
      %nvhpc
  concretizer:
    unify: true
  config:
    build_jobs: 4
```

spec の各フラグが必要な理由：

- +cuda cuda_arch=80 : A100 向け sm_80。hypre・mfem に自動伝播。
- precision=single : mfem・hypre にも伝播。FWI で標準。
- +gpu_aware_mpi : HPC-X OpenMPI が CUDA-aware でビルド済みなため有効化可能。hypre の +gpu-aware-mpi にも伝播。
- cxxflags=="-noswitcherror" cflags=="-noswitcherror" : yaml-cpp が gcc 専用フラグ -Weffc++・-pedantic-errors を nvcc++ に渡してエラーになるため、nvcc++ に「未知のフラグは無視するモード」を全パッケージ伝播 (==は sticky propagation)。
- ldlibs=="-lstdc++fs" : rhel8 の gcc 8.5 は std::filesystem を独立アーカイブ libstdc++fs に分離している。adios2 (gcc 9+ で filesystem を libstdc++ に統合する前提で書かれている) のリンクが通るよう明示的に追加。
- ^hypre@2.33 : hypre 3.x は ES の gcc 8.5 環境で cstddef + extern "C" 衝突、CMake の FindMPI 非互換、UMPIRE 経由のさらなる依存といった連鎖トラブルがあるため、2.33 系を明示的に固定。shadow recipe (spack-repo/packages/hypre/package.py) が cuSOLVER を無効化することで hypre 内部 FSAI の単精度バグも回避する。

- `%nvhpc` : semsws 本体を `nvc++` でビルド。openblas や netcdf-c など gcc 強制パッケージとは独立。

■packages_gpu.yaml 内の各エントリの根拠

- `nvhpc` : `prefix` を `/opt/share/NVIDIAHPCSDK/24.7` に設定。**compilers/サブディレクトリは含めない**。Spack の `nvhpc` パッケージは `<prefix>/Linux_x86_64/<version>/compilers/lib` を内部で付加するため、`prefix` を `.../compilers` にすると重複し `libblas` が見つからなくなる。
- `openmpi` : NVHPC 同梱 HPC-X 4.1.7。external 登録のみで `+cuda fabrics=ucx` と書いておく。
- `cuda` : `/opt/share/CUDA/12.0.0` を指す。NVHPC 同梱 CUDA 12.5 は分割レイアウトで `cusparse.h` が `math_libs` 下にあり、MFEM の `find_package(CUDAToolkit)` が失敗する。
- `hdf5` / `metis` / `netcdf-c` / `netcdf-cxx4` / `netcdf-fortran` / `openblas` はすべて `require: '%gcc'` で Spack ビルド。理由は `nvc` の `autotools probes` が `-m32` や `-Wno-error` で詰まること、およびシステム `module` が `serial` 限定で `+mpi` と整合しないこと。gcc でビルドした C ライブラリを `nvc++` ビルドの `mfem` にリンクするのは `libstdc++/libc` 共通で ABI 問題なし。

■インストール実行

```
spack concretize -f
spack install --fail-fast 2>&1 | tee install_gpu.log
```

- `spack concretize -f` : env 内の `spack.yaml` から `spack.lock` を再生成。-f は強制再解決。
- `--fail-fast` : 依存パッケージのどれかが失敗した瞬間に停止。
- フォアグラウンドで実行すると進捗 UI ([+] [-] [x] のアニメーション) が見えるので `tmux` 内で走らせるのを推奨。

ビルド時間目安 (初回フル) : 50~90 分。最大は `hydre + cuda kernel` の `nvcc compile` で 30 分以上を要する。

■CPU ビルド環境 CPU 側は別 env として並立させる。NVHPC をロードせず system OpenMPI 4.0.5 を使う構成 :

```
spack env deactivate          # GPU envから抜ける
source ~/load_cpu.sh         # CPU用モジュールに切り替え
spack compiler find /usr/bin # gcc@8.5を確認
spack env create semsws-cpu
spack env activate semsws-cpu
spack config edit
# packages_cpu.yamlの内容を貼り、specsには
# - semsws@main precision=single ~hydre@2.33 %gcc
# を入れる
spack concretize -f
spack install --fail-fast 2>&1 | tee install_cpu.log
```

CPU env と GPU env の主な差分：

- nvhpc / cuda 外部宣言を削除、代わりに gcc が唯一のコンパイラ。
- openmpi は/opt/share/OpenMPI/4.0.5 を指す external。
- spec から +cuda / +gpu_aware_mpi / cxxflags=="-noswitcherror" を削除。%gcc に変更。
- ^hype02.33 は念のため残す (CMake FindMPI の問題は CPU 側でも発生し得るため)。

■CPU・GPU の切り替え

GPU版を使う

```
source ~/load_gpu.sh && spack env activate semsws-gpu && spack load semsws
```

CPU版を使う

```
source ~/load_cpu.sh && spack env activate semsws-cpu && spack load semsws
```

source ~/load_*.sh を毎回入れる必要があるのは、GPU 用と CPU 用でロードする system module が排他的 (NVIDIAHPCSDK と system OpenMPI は同時にロードできない) なため。Spack env の activate/deactivate だけでは module 環境は切り替わらない。

2.5.5 ES ジョブスクリプト例 (Spack ビルド版)

ES のバッチジョブシステム (PJM) で Spack インストール済みの semsws を実行する例。GPU env と CPU env で rscgrp・load_*.sh・spack env がそれぞれ異なる。

■GPU env (A100 ノード、1 ノード × 8 GPU) .

```
#!/bin/bash
#PJM -L rscgrp=gpu
#PJM -L node=1
#PJM -L elapse=01:00:00
#PJM -j

# モジュール環境と Spack env をロード
source ~/load_gpu.sh
spack env activate semsws-gpu
spack load semsws

# 1 ノード上で 8 ranks (A100×8 にそれぞれバインド)
mpirun -np 8 -map-by ppr:8:node \
    semsws --config config.yaml
```

■CPU env (CPU ノード、1 ノード × 16 ranks) .

```
#!/bin/bash
#PJM -L rscgrp=cpu
#PJM -L node=1
#PJM -L elapse=04:00:00
#PJM -j

source ~/load_cpu.sh
spack env activate semsws-cpu
spack load semsws
```

```
mpirun -np 16 semsws --config config.yaml
```

■複数ノード. 複数ノード使う場合は#PJM -L node=Nに変更し、`mpirun -np` を合計 rank 数 (GPU:N*8、CPU:N*16 など) に揃える。PJM の `NODE_LIST` 環境変数等は `mpirun` が自動で拾う。

■ジョブ投入.

```
pjsub job_gpu.sh          # 投入
pjstat                    # キュー状況確認
pjcat <jobid> -o          # stdoutを表示
```

2.5.6 Spack 環境での pytest 実行

```
# envをactivateしてsemswsバイナリのprefixを取得
spack env activate semsws-gpu
spack load semsws
SEMSWS_PREFIX=$(spack find --paths semsws | \
    awk 'NR>1 && /\// {print $NF; exit}')

# pytest はテストディレクトリへ cd せず、絶対パスで指定する
pytest -v --build-dir $SEMSWS_PREFIX --np 4 --device cuda \
    $WORK/SEMSWS/test/consistency/
```

- `--build-dir`:pytest が `semsws` バイナリを探す prefix。Spack install では `$prefix/bin/semsws` に置かれる。
- `--np`:MPI プロセス数。各 SEMSWS テストはメッシュサイズが小さく、過剰な np (128 以上等) では METIS 分割で空 rank が発生し `IsotropicElasticMaterial3D::AttenuationQmu()` 等でセグフォルトする。安全な目安は np=4~16。
- `--device cuda / cpu`:シミュレーションのデバイス選択。ES では GPU env なら `cuda`、CPU env なら `cpu` を渡す。
- `SEMSWS_PREFIX` を `spack find --paths` で取るのは `spack location -i` と違って内部的に `git` を呼ばないのでバッチジョブ環境でも失敗しないため。

2.5.7 2 回目以降のセッション (LUMI/ES 共通)

```
# LUMI
source ~/load.sh && spack load semsws

# ES GPU
source ~/load_gpu.sh && spack env activate semsws-gpu && spack load semsws

# ES CPU
source ~/load_cpu.sh && spack env activate semsws-cpu && spack load semsws
```

`which semsws` で `PATH` に入ったことを確認してから `srun` (LUMI) または `mpirun` (ES) でジョブを投入する。

2.6 ビルド結果の確認

ビルドが成功すると、`build/src/`以下に実行ファイルが生成される。

表 2.5 生成される実行ファイル

実行ファイル	説明
<code>semsws</code>	シミュレーション本体
<code>partition_mesh</code>	メッシュの事前分割
<code>refine_mesh</code>	メッシュの細分化
<code>evaluate_mesh</code>	メッシュ品質の評価
<code>combine_mesh</code>	分割メッシュの結合（可視化用）
<code>bp2gf</code>	ADIOS2 BP を GLVis 形式に変換

ビルドが正常に完了したか確認するには：

```
./build/src/semsws --help
```

テストでの検証も可能である。テストの実行には Python 3.9 以上と `pytest`、`numpy` が必要である。

```
# 全テスト実行 (defaultはCPU、serial run)
pytest test/ --build-dir ./build -v

# MPI並列・GPUデバイス指定
pytest test/ --build-dir ./build --np 4 --device cuda -v

# 波形検証テストのみ
pytest test/waveform/ --build-dir ./build -v

# 出力形式の整合性テストのみ
pytest test/consistency/ --build-dir ./build -v
```

用意されているテストは以下の2種類である。

■**波形検証テスト** (`test/waveform/`) シミュレーション結果をリファレンス波形と比較し、正規化 L2 ノルムが閾値以内であることを検証する。ケース名に `analytic` を含むものは解析解との比較、それ以外はすべて SPECfem との比較である。以下のケースが含まれる：

表 2.6 波形検証テストケース

次元	物理モデル
2D	acoustic, elastic, acoustic_visco, elastic_visco
3D	acoustic_analytic, elastic, elastic_analytic, elastic_visco

--keep-results オプションを付けて実行すると、各テストディレクトリ(例:test/waveform/3D/elastic/)の results/ にシミュレーション結果が保存され、同ディレクトリ内の ref/ に置かれたリファレンス波形とファイル単位で見比べることができる：

```
pytest test/waveform/ --build-dir ./build --keep-results -v
```

■出力形式整合性テスト (test/consistency/) 受振器の出力形式 (ASCII、HDF5、SU) 間で波形が数値的に一致することを検証する。

2.7 GLVis のインストール (推奨)

GLVis は MFEM プロジェクトが公開している軽量な可視化ツールである^{*2}。SEMSWS が results/wavefield/ に出力する MFEM 形式のメッシュとグリッド関数ファイルを、そのまま読み込んで実行中あるいは実行後の波動場をフレーム単位で確認することができる。SEMSWS 自体の動作には必須ではないが、インストールしておくこと第 3 章の Quick Start チュートリアルが格段に確認しやすくなる (プロットスクリプトを書く前に、コマンド一発でシミュレーションの動作確認ができる)。

2.7.1 ビルド

GLVis は前節でインストールした MFEM をそのまま参照できる。

```
cd ~/program_ubuntu
git clone https://github.com/GLVis/glvis.git
cd glvis
make -j MFEM_DIR=../mfem
```

生成される実行ファイルはチェックアウト直下の glvis である。PATH に追加するか、フルパスで呼び出せる状態にしておく。

SDL・OpenGL・fontconfig・freetype のシステムパッケージが必要。Debian / Ubuntu の場合：

```
sudo apt install libsdl2-dev libglew-dev libfontconfig-dev \
    libfreetype-dev
```

2.7.2 基本的な使い方

メッシュ単体、あるいは SEMSWS が出力したメッシュ+グリッド関数の組を指定して起動する：

^{*2} <https://glvis.org/>

```
# メッシュのみ表示
glvis -m mesh.mesh

# メッシュ+1フレームの波動場スナップショット
glvis -m results/wavefield/mesh.mesh -g results/wavefield/sol_000500.gf

# 分割 (MPI並列) メッシュ - N は出力ランク数
glvis -np 2 -m results/wavefield/mesh -g results/wavefield/sol_000500
```

分割メッシュ形式では、GLVisがmesh.000000、mesh.000001、...と対応するsol_000500.000000...を自動的に読み込み、1つのウィンドウに再結合して表示する。

その他の起動オプションやウィンドウ操作のキーバインドはGLVisのREADME^{*3}を参照のこと。

^{*3} <https://github.com/GLVis/glvis>

第 3 章

クイックスタート

この章では、2次元音響波シミュレーションを例に、設定ファイルの記述からシミュレーションの実行、結果の可視化までの一連の流れを説明する。使用する例題は `examples/visualization/2D/acoustic/` に含まれている。

3.1 設定ファイル

SEMSWS のシミュレーションは、すべてのパラメータを 1 つの YAML ファイル (`config.yaml`) で記述する。以下に本例題の設定ファイルをセクションごとに説明する。

3.1.1 シミュレーション基本設定

```
simulation:
  dimension: 2
  order: 4

  time:
    steps: 10000
    dt: 0.0001
```

`dimension` は空間次元 (2 または 3)、`order` は多項式次数 (スペクトル要素法の次数) を指定する。`time` セクションで時間ステップ数と時間刻み幅 Δt を設定する。本例題では $\Delta t = 0.0001\text{s}$ で 10000 ステップ、すなわち 1 秒間のシミュレーションとなる。

3.1.2 メッシュ

```
mesh:
  type: internal
  origin: [0.0, 0.0]
  size: [1000.0, 500.0]
  elements: [50, 25]
  save: true
  max_freq: 75.0
  ppw: 5.0
```

`type: internal` は SEMSWS 内部で regular/structured メッシュを自動生成する設定である。`size` でドメインサイズ (m)、`elements` で要素数を指定する。本例題では $1000\text{ m} \times 500\text{ m}$ の領域を 50×25 要素で分割する。`save` を指定すると `results` ディレクトリに `glvis` 可視化用 `mesh` が生成される。`max_freq`、`ppw` はメッシュの品質評価用のパラメータでそれぞれ最大周波数と波長に対するサンプリング数 (points per wavelength)。`summary.txt` での出力やのちに説明する `evaluate_mesh` プログラムに使用する。

外部メッシュ (GMSH、CUBIT/ExodusII 等) を使用する場合は `type: external` とし、`file:` にファイルパスを指定する。

3.1.3 物性

```
material:
  type: isotropic_acoustic
  format: constant
  vp: 1500.0
  rho: 1000.0
  attenuation:
    enabled: false
```

均質な音響媒質を定義している。`type` で物理モデル (`isotropic_acoustic`) を選択し、`format: constant` で均質モデルであることを指定する。P 波速度 1500 m/s 、密度 1000 kg/m^3 の設定である。

3.1.4 境界条件

```
boundary:
  absorbing:
    type: cerjan
    sides: []
    thickness: 100.0
    alpha: 0.4

  dirichlet:
    attributes: [top, right, left, bottom]
```

`dirichlet` で全辺に Dirichlet (固定端) 境界条件を設定している。吸収境界 (Cerjan スポンジ層) は `sides: []` で無効化されている。吸収境界を有効にする場合は `sides: [top, right, left, bottom]` のように適用面を指定する。海域での反射法地震探査や屈折法地震探査のシミュレーションでは、海面を Dirichlet に設定する必要がある (`dirichlet: [top]`)。

3.1.5 震源

```
sources:
  list:
    - id: 1
      name: "Source 1"
```

```
type: pressure
location: [500.0, 400.0]
wavelet:
  type: ricker
  frequency: 30.0
  amplitude: 1.0e10
  delay: 0.05
```

ドメイン中央付近 (500 m, 400 m) に 30 Hz の Ricker ウェーブレットを持つ圧力点震源を 1 つ配置している。delay はウェーブレットのピーク到達時刻 (秒) である。

3.1.6 受振器

```
receivers:
  output:
    format: ascii
    type: [PS]

  list:
    - name: "R001"
      location: [400.0, 300.0]
    - name: "R002"
      location: [500.0, 300.0]
    - name: "R003"
      location: [600.0, 300.0]
```

3 点の受振器を配置し、圧力スカラー場 (PS) を記録する。出力形式は `ascii` で、1 列目が時刻、2 列目が振幅のテキストファイルとなる。format では他にも `hdf5` と `su` がしていできる。su は seismic unix 用のフォーマット。

3.1.7 波動場出力

```
simulation:
  output:
    wavefield:
      enabled: true
      interval: 1000
      fields: ["PS"]
      formats:
        - type: "glvis"
        - type: "paraview"
          refinement: 4
          data_format: "binary32"
        - type: "gmt"
          resolution: [100, 50]
```

1000 ステップごとに波動場スナップショットを出力する。出力形式は 3 種類同時に指定できる：

- **GLVis** — MFEM ネイティブ形式。glvis コマンドで対話的に可視化
- **ParaView** — VTU 形式。ParaView で 3 次元可視化に適する
- **GMT** — テキストグリッド形式 (x y value)。resolution でグリッド解像度を指定

3.1.8 デバイス設定

```
device:
  type: cpu
```

cpu、cuda、hip から選択する。GPU ビルドの場合でも cpu を指定すれば CPU 上で実行される (ただしかなり遅い、cpu 実行の時は CPU ビルドを使用すべき)。

3.2 シミュレーションの実行

設定ファイルのあるディレクトリで以下のコマンドを実行する：

```
cd examples/visualization/2D/acoustic

# 4プロセスで実行
mpirun -np 4 ../../../../build/src/semsws \
  --config config.yaml
```

同ディレクトリに含まれる run.sh を使うこともできる：

```
bash run.sh 4      # 引数はMPIプロセス数
```

実行が完了すると、results/ディレクトリに以下が生成される：

表 3.1 出力ファイル一覧

パス	説明
summary.txt	シミュレーション情報 (物性、メッシュ、性能等)
R001_0001.p 等	受振器波形 (ASCII テキスト)
wavefield/gmt/	GMT スナップショット
wavefield/paraview/	ParaView 用 VTU ファイル
wavefield/glvis/	GLVis 用バイナリファイル
material/	物性分布の出力
mesh.mesh	使用したメッシュ

3.3 結果の確認

3.3.1 サマリー

results/summary.txt にはシミュレーションの要約が記録される。以下に本例題の出力例を示す：

```
Discretization:
  Order: 4
  Elements: 1250
  DOFs: 20301

Time stepping:
  dt: 0.0001 s
  nt: 10000
  Simulation time: 1 s

Numerical stability:
  h_min: 20.00 m, h_max: 20.00 m
  CFL dt_max: 7.3535e-04 s (dt/dt_max = 0.14)
  PPW: 12.5 (f=30.0 Hz)

Performance:
  Setup time: 0.18 s
  Run time: 2.27 s
  Total time: 2.44 s
  DOFs/sec: 8.96e+07
```

特に CFL dt_max が設定した Δt より大きいことを確認する。 $\Delta t > \Delta t_{\max}$ の場合、計算は不安定となり発散する。PPW (Points Per Wavelength) は 1 波長あたりのグリッド点数であり、5 以上が一般的な目安である。

3.3.2 波動場スナップショット

付属の Python スクリプトで波動場を可視化できる：

```
python3 scripts/plot/plot_wavefield_2d.py --results-dir ./results
```

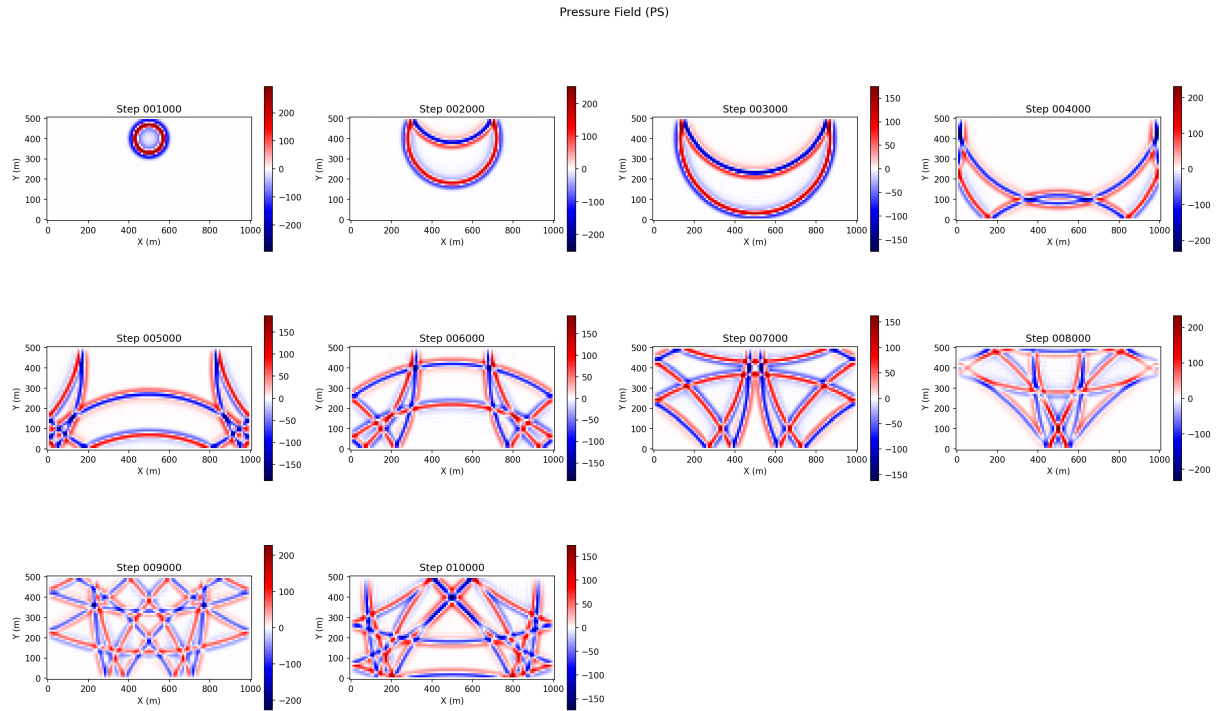


図 3.1 圧力場スナップショット（1000 ステップ間隔）。

本例題ではすべての辺に Dirichlet 境界条件を設定しているため、境界面で波が反射している様子が確認できる。実際の地震波シミュレーションでは吸収境界条件を使用して人工的な反射を抑制する。

3.3.3 受振器波形

受振器の波形ファイルは 2 列の ASCII テキスト形式である。1 列目が時刻（秒）、2 列目が振幅（圧力）である。gnuplot や Python など容易にプロットできる。

第 4 章

入力ファイルリファレンス

SEMSWS のシミュレーションは 1 つの YAML ファイル (config.yaml) ですべてのパラメータを指定する。本章では設定ファイルの全オプションを網羅的に記述する。基本的な使い方については第 3 章を参照されたい。

4.1 YAML 形式の基礎

設定ファイルは YAML (YAML Ain't Markup Language) 形式で記述する。以下に YAML の基本的な構文規則を示す。

- スペースで階層構造を表現する。
- `key: value` の形式でキーと値を記述する
- リストは `-` で表現する
- `#`以降はコメントとして無視される
- 文字列は引用符なしでも記述できるが、特殊文字を含む場合は `"..."` で囲む

```
# スカ ラー 値
name: "My Simulation"
order: 4
dt: 0.0001

# ネストされた構造
simulation:
  dimension: 2
  time:
    steps: 10000
    dt: 0.0001

# リスト
sides: [bottom, right, top, left]

# リスト (複数行形式)
list:
  - name: "R001"
    location: [400.0, 300.0]
```

```
- name: "R002"
  location: [500.0, 300.0]
```

4.2 厳密キー検証 (Strict-Key Validation)

YAML ロードは設定ファイルの各ブロックを既知キー集合と照合する。どこかに未知のキー（タイプミス・リネーム済みの旧キー・古い設定ファイルからコピペされたサブブロックなど）があると、それを無言で無視せず、以下のような形式で実行時エラーとして中断する：

```
Error validating config: Unknown key `<bad_key>` in <section>
(allowed: <そのブロックで有効なキーの一覧>)
```

同じ機構が、削除された旧キーを個別のメッセージで拒否する：

- `boundary.free_surface` は削除された。弾性領域の自由表面は自然境界条件なのでブロックごと削除、音響領域の自由表面（圧力 = 0）は `boundary.dirichlet.attributes` で宣言する。
- `simulation.output.wavefield.format` と `receivers.output.format`（単数形）は `formats`：（マッピング要素の複数形リスト）に置き換え済。§4.5.5 / §4.10.1 参照。
- 出力フォーマット要素内の `component`：<int>（単数形）は `components`：[<int>, ...] に置き換え済。
- `simulation.checkpoint_storage`：<host_memory> は `memory` にリネーム済。
- `material.format`：<ascii> は `grid` にリネーム済。

古い設定ファイルを移植する場合は、上記のリネームを適用し、本章に記載されていないキーは削除すること。

4.3 設定ファイルの全体構造

設定ファイルの全体構造は以下の通りである：

```
name: "... "           # シミュレーション名（任意）
description: "... "     # 説明（任意）
simulation:             # シミュレーション基本設定・出力
mesh:                  # メッシュ
material:               # 物性
boundary:               # 境界条件
sources:                # 震源
receivers:              # 受振器
device:                 # 実行デバイス
```


4.4 ルートレベルパラメータ

キー	型	必須	デフォルト	説明
name	string	任意	"Seismic wave simulation"	シミュレーション名。 summary.txt に記録される
description	string	任意	"No description provided."	自由記述

4.5 simulation セクション

4.5.1 基本パラメータ

キー	型	必須	値	説明
dimension	int	必須	2, 3	空間次元
order	int	必須	≥ 1	多項式次数 (GLL 点数 = $N + 1$)

GPU 実装では共有メモリ制約により order に上限がある。

4.5.2 時間積分 (time)

キー	型	必須	デフォルト	説明
time.dt	real	必須	—	時間刻み幅 (s)
time.steps	int	必須	—	時間ステップ数
time.cfl_factor	real	必須	—	CFL 安全係数 (0–0.7)
time.t0	real	任意	0.0	シミュレーション開始時刻 (s)

Δt は CFL 条件 ($\Delta t < \Delta t_{\max}$) を満たす必要がある。cfl_factor はメッシュ品質評価に使用される。総シミュレーション時間は $\text{steps} \times \Delta t$ で決まる。

4.5.3 シミュレーションモード

キー	型	必須	デフォルト	説明
mode	string	任意	forward	forward, inversion, misfit_only
misfit_type	string	任意	l2_waveform	l2_waveform, normalized_correlation
num_checkpoints	int	任意	10	Revolve チェックポイント数
checkpoint_storage	string	任意	"memory"	"disk", "memory"
checkpoint_device	string	任意	"auto"	"host", "device", "auto"
checkpoint_dir	string	条件	"./checkpoints/"	disk 時に必須 (未対応)
kernel_output_dir	string	任意	"./kernels/"	感度カーネル出力先

forward が標準的なフォワードシミュレーションである。inversion および misfit_only は Full Waveform Inversion (FWI) 用であり、それぞれ checkpoint アルゴリズムと adjoint を使用した勾配計算、そして波形の誤差計算ができる。現在は 2 次元音響波でのみ対応している。その他のキーも FWI 用であり、まだ十分なデバッグは終了していない。

4.5.4 出力設定 (output)

キー	型	必須	デフォルト	説明
output.directory	string	任意	"./"	出力ルートディレクトリ
output.summary_file	string	任意	" "	サマリーファイル名 (空文字で無効)
output.log_interval	int	任意	100	ログ出力間隔 (ステップ数)

全ての出力ファイルは output.directory で指定したディレクトリに出力される。output.summary_file には計算の要約が出力される。便利なので、適当に summary.txt などと指定しておけばいい。空文字でなにも出力しなくなる。output.log_interval は live で何ステップごとに計算の進行状況が確認するかを指定する。並列計算の際は MPI 通信がその都度発生するので、適切なサイズに設定する。特に GPU 計算の際は Host/Device での通信が都度発生するので気を付ける。

4.5.5 波動場出力 (output.wavefield)

キー	型	必須	デフォルト	説明
enabled	bool	任意	false	波動場スナップショット出力を有効化
interval	int	条件	—	出力間隔 (ステップ数)
fields	string 配列	任意	["DISP"]	出力フィールド
formats	配列	条件	—	出力フォーマットのリスト

`fields` に指定可能な値: DISP (変位)、VEL (速度)、ACC (加速度)、PS (圧力スカラー、acoustic wave simulation のときのみ)。

■出力フォーマット `formats` 配列の各要素に以下のオプションを指定する:

キー	対象	デフォルト	説明
type	全て	—	glvis, paraview, gmt
refinement	paraview	1	分割レベル
data_format	paraview	binary32	ascii, binary (64bit) , binary32 (32bit)
compression	paraview	-1	zlib 圧縮レベル (0-9, -1=デフォルト、おそらく zlib 付きで mfem のビルド必須、未テスト)
resolution	gmt	—	出力グリッド数 (解像度) [nx, ny]
components	gmt	[-1]	-1=magnitude, 0=x, 1=y, 2=z (複数指定可 ex. [-1, 0, 1, 2])
cross_sections	gmt (3D)	—	断面位置

3次元 GMT 出力では `cross_sections` で断面位置を指定する:

```
cross_sections:
  yz: [2500.0]      # x=2500mでのyz断面
  xz: [2500.0]      # y=2500mでのxz断面
  xy: [1000.0]      # z=1000mでのxy断面
```

■サイドごとのオーバーライド (流体-固体連成用)。 `material.type: coupled` を使う場合、流体側と固体側は別々の `ParSubMesh` に住んでおり、波動場出力ファイルもサイドごとに分けて書き出される。 `wavefield`: ブロック内に `fluid`: や `solid`: のサブブロックを追加すると、`enabled` / `interval` / `fields` / `formats` / `components` をそのサイドだけ上書きできる。指定しないキーは親の `wavefield`: 設定を継承する。片側で `enabled: false` とすると、そのサイドだけ出力を停止し、もう一方は通常通り書き出す。

```
wavefield:
  enabled: true
  interval: 500
```

```

formats:
  - type: glvis
fluid:
  fields: [PS]          # 流体側は圧力のみ
solid:
  fields: [DISP, VEL]   # 固体側は変位 + 速度

```

4.5.6 物性値出力 (output.material)

キー	型	必須	デフォルト	説明
enabled	bool	任意	false	物性値出力を有効化
fields	string 配列	任意	["vp"]	出力フィールド
formats	配列	条件	—	フォーマット（波動場と同じ）

fields に指定可能な値：vp, vs, rho, qkappa, qmu。物性値出力はシミュレーション開始時に 1 回のみ書き出される。

material.type: coupled のシミュレーションでは、上記の波動場と同様に output.material 配下にも fluid: / solid: サブブロックを配置して、サイドごとに enabled / fields / formats を上書きできる（固体側だけ vs を出す等）。

4.5.7 simulation セクションの設定例

```

simulation:
  dimension: 3
  order: 4

  time:
    steps: 20000
    dt: 0.00005
    cfl_factor: 0.3

  output:
    directory: "./results"
    summary_file: "summary.txt"
    log_interval: 500

  wavefield:
    enabled: true
    interval: 1000
    fields: ["DISP"]
    formats:
      - type: "paraview"
        refinement: 2
        data_format: "binary32"

```

```

- type: "gmt"
  resolution: [200, 200]
  components: [-1, 0, 1, 2]
  cross_sections:
    xy: [0.0]
    xz: [2500.0]

material:
  enabled: true
  fields: ["vp", "vs", "rho"]
  formats:
    - type: "paraview"
      refinement: 2

```

4.6 mesh セクション

4.6.1 共通パラメータ

キー	型	必須	デフォルト	説明
type	string	必須	—	internal, external, partitioned
max_freq	real	任意	—	メッシュ品質評価用の最大周波数 (Hz)
ppw	real	任意	—	Points Per Wavelength
save	bool	任意	false	メッシュを results に保存

4.6.2 内部メッシュ (type: internal)

SEMSWS 内部で構造格子メッシュを自動生成する。

キー	型	必須	デフォルト	説明
origin	real 配列	必須	—	原点座標 [x,y] or [x,y,z] (m)
size	real 配列	必須	—	ドメインサイズ [Lx,Ly] or [Lx,Ly,Lz] (m)
elements	int 配列	必須	—	要素数 [Nx,Ny] or [Nx,Ny,Nz]
partition	string	任意	metis	metis or cartesian
partition_grid	int 配列	条件	—	cartesian 時の分割数 [nx,ny,nz]
attr_y_threshold	real	任意	—	指定すると y 座標 (2D) / z 座標 (3D) がこの値より小さい要素に attribute = 1、大きい要素に attribute = 2 を付与。流体-固体連成メッシュの簡易セットアップに使う。

`partition: cartesian` は METIS 不要の構造的分割である。`partition_grid` では三次元で `[4, 4, 2]` と指定すると `x, y, z` 軸方向にそれぞれ 4, 4, 2 と分割される。このとき実行コマンドの並列数 (`-np 4` など) は 32 (積) と合ってなければいけない。

4.6.3 外部メッシュ (type: external)

キー	型	必須	デフォルト	説明
<code>file</code>	string	必須	—	メッシュファイルパス
<code>format</code>	string	任意	自動検出	<code>gms</code> , <code>mfem</code> , <code>exodus</code> , <code>vtk</code>
<code>partition</code>	string	任意	<code>metis</code>	MPI 分割方法

ExodusII 形式 (CUBIT) を使用する場合、MFEM のビルド時に `MFEM_USE_NETCDF=ON` が必要である。

4.6.4 事前分割メッシュ (type: partitioned)

`partition_mesh` ツールで事前分割した MFEM フォーマットのメッシュを読み込む。各 MPI ランクが自身の分割ファイルのみを読む。大規模メッシュや何度も同じメッシュで計算する際に有用。

キー	型	必須	デフォルト	説明
<code>partitioned.directory</code>	string	必須	—	分割ファイルのディレクトリ
<code>partitioned.nparts</code>	int	必須	—	分割数 (MPI プロセス数と一致)

ファイル命名規則は `mesh.mesh.000000`, `mesh.mesh.000001`, ... である。

4.6.5 mesh セクションの設定例

内部メッシュ:

```
mesh:
  type: internal
  origin: [0.0, 0.0, 0.0]
  size: [5000.0, 5000.0, 3000.0]
  elements: [50, 50, 30]
  max_freq: 20.0
  ppw: 5.0
  save: true
  partition: cartesian
  partition_grid: [4, 4, 2]
```

外部メッシュ:

```
mesh:
  type: external
```

```
file: "./meshes/model.exo"
max_freq: 15.0
ppw: 5.0
```

事前分割メッシュ:

```
mesh:
  type: partitioned
  partitioned:
    directory: "./meshes/partitioned"
    nparts: 32
  max_freq: 15.0
  ppw: 5.0
```

4.7 material セクション

4.7.1 物性タイプとフォーマット

type の値	説明
isotropic_acoustic	等方性音響媒質 (vp, rho)
isotropic_elastic	等方性弾性媒質 (vp, vs, rho)
coupled	流体 – 固体連成媒質。流体側は isotropic_acoustic、固体側は isotropic_elastic。fluid: と solid: のサブブロックで両者を独立に宣言する。詳細は §4.7.8。

format の値	説明
constant	YAML 内で均質物性を直接指定
grid	構造格子の ASCII ファイル
by_attribute	メッシュ属性ごとのテキストファイル
by_attribute_mixed	属性ごとに constant/grid を混合指定する YAML ファイル
adios2	ADIOS2 BP ファイル (FWI 用)

type: coupled の場合は最上位で format を指定しない。ネストされる fluid: / solid: サブブロックがそれぞれ独自の format を持つ。

4.7.2 format: constant

YAML 内で物性値を直接指定する。

キー	型	elastic	acoustic
vp	real	必須	必須
vs	real	必須	—
rho	real	必須	必須

単位はそれぞれ m/s, m/s, kg/m³。

```
material:
  type: isotropic_elastic
  format: constant
  vp: 6000.0
  vs: 3500.0
  rho: 2700.0
```

4.7.3 format: grid

構造格子上の不均質物性を ASCII ファイルから読み込む。file にファイルパスを指定する。

ファイルフォーマット：

```
Nx Ny [Nz]          # 格子点数
dx dy [dz]          # 格子間隔 (m)
x0 y0 [z0]          # 原点 (m)
vp vs rho [Qkappa Qmu] # 各格子点の物性値
vp vs rho [Qkappa Qmu] # (Nx*Ny[*Nz]行)
...
```

弾性の場合は5列 (vp, vs, rho, Qkappa, Qmu)、音響の場合は3列 (vp, rho, Qkappa)。Q 値の列は減衰が有効な場合のみ必要。

データは lexicographic order (x 方向が最も速く変化) で格納する。インデックスと座標の対応は以下の通り：

■2次元 i 行目のデータは格子点 (i_x, i_y) に対応する。

$$i = i_y \times N_x + i_x$$

$$x = x_0 + i_x \times dx, \quad y = y_0 + i_y \times dy$$

すなわち、 x 方向に N_x 点走査してから y 方向に1つ進む。

■3次元 i 行目のデータは格子点 (i_x, i_y, i_z) に対応する。

$$i = i_z \times N_x \times N_y + i_y \times N_x + i_x$$

$$x = x_0 + i_x \times dx, \quad y = y_0 + i_y \times dy, \quad z = z_0 + i_z \times dz$$

すなわち、 $x \rightarrow y \rightarrow z$ の順で走査する。グリッド外の点は最近傍の境界値にクランプされる。

原点 (x_0, y_0) または (x_0, y_0, z_0) はグリッドの最小座標点であり、通常はメッシュの origin と一致させる。**semsws の座標系では y 軸 (2D) / z 軸 (3D) が上向き正であるため、原点はドメインの最深部に位置する。したがってファイルの先頭行は最も深い場所の物性値から始まる。**

4.7.4 format: by_attribute

メッシュの要素属性 (attribute) ごとに物性を定義するテキストファイル。外部メッシュ (CUBIT 等) で属性を付与した場合に使用する。

#	attr	vp	vs	rho	[Qkappa	Qmu]
1		6000.0	3500.0	2700.0	100.0	100.0
2		3000.0	2000.0	2200.0	50.0	50.0

4.7.5 format: by_attribute_mixed

属性ごとに constant (均質) または grid (格子ファイル) を混合指定する外部 YAML ファイル。

```
attributes:
  1:
    mode: constant
    vp: 5500.0
    vs: 3175.0
    rho: 2800.0
  2:
    mode: grid
    file: material_layer2.dat
```

4.7.6 format: adios2

FWI の反復で生成された GLL 点上の物性データを ADIOS2 BP 形式で読み込む。semsws_export_model ツール (§??相当) で出力した BP ファイルをそのまま入力できる。

キー	型	必須	説明
vp_file	string	必須	Vp の BP ファイルパス
vs_file	string	条件付	Vs の BP ファイルパス (type: isotropic_elastic で必須)
rho_file	string	必須	ρ の BP ファイルパス
qkappa_file	string	任意	Q_κ の BP ファイルパス (粘性音響・粘弾性で必要)
qmu_file	string	任意	Q_μ の BP ファイルパス (粘弾性で必要)

type: coupled の fluid: / solid: サブブロックでは adios2 フォーマットは現在サポートされていない。

4.7.7 減衰 (attenuation)

一般化 Zener モデル (Standard Linear Solid: SLS) による減衰を設定する。

キー	型	必須	デフォルト	説明
<code>attenuation.enabled</code>	bool	任意	false	減衰の有効化
<code>attenuation.f0</code>	real	条件	1.0	参照周波数 (Hz)。enabled=true で必須
<code>attenuation.n_units</code>	int	条件	3	SLS 緩和機構の数。enabled=true で必須
<code>attenuation.Qkappa</code>	real	条件	—	体積弾性率の Q 値 (format: constant 時に必須)
<code>attenuation.Qmu</code>	real	条件	—	せん断弾性率の Q 値 (format: constant かつ弾性で必須)
<code>attenuation.Qkappa_file</code>	string	任意	—	Q_κ のグリッドファイル (format: grid/by_attribute で Q をファイル指定する場合)
<code>attenuation.Qmu_file</code>	string	任意	—	Q_μ のグリッドファイル (弾性で同上)

Q 値のフィッティング帯域は $[0.1f_0, 10f_0]$ である。format: grid や by_attribute 形式では通常ファイル内の Q 値列がそのまま使われるため、上の YAML キーは不要。明示的に別ファイルから Q を読み込みたい場合に Qkappa_file/Qmu_file を指定する。

4.7.8 type: coupled (流体 – 固体連成)

流体（音響）領域と固体（弾性）領域が界面を共有するシミュレーションでは、material.type: coupled を指定する。この場合、最上位の material: ブロックは fluid: と solid: の二つのサブブロックを持ち、それぞれが独自の type・format・物性パラメータを宣言する（両者は独立であり、例えば流体を constant、固体を grid にすることもできる）。

キー	型	必須/オプション	説明
attribute	int	必須	このサブ領域を識別する親メッシュ要素の属性値。 fluid と solid で異なる値にすること。
type	enum	必須	fluid では isotropic_acoustic、solid では isotropic_elastic を指定。内部で再度 coupled を指定することはできない。
format	enum	必須	通常材料と同じ値: constant、grid、 by_attribute、by_attribute_mixed。
vp, vs, rho	real	条件付	format: constant の場合に必須（弾性側は vs も 必須）。
file	path	条件付	format: grid / by_attribute / by_attribute_mixed の場合に必須。
attenuation	block	オプション	attenuation: サブブロックを持つ場合、非連成材 料と同じキー構成 (§4.7.7) を用いる。fluid: 側で 有効化すると粘性音響、solid: 側で有効化すると粘 弾性となる。

流体 – 固体界面の境界属性は実行時に FluidSolidInterface::Setup が自動検出する（二つの ParSubMesh のいずれか一方にのみ存在する境界属性のうち、親メッシュが元々持つ外側境界には含まれないものが界面として抽出される）。YAML 側で明示的に指定する必要はない。

■連成サブブロックで使用可能な format. fluid: / solid: サブブロックで指定できる format は constant / grid / by_attribute / by_attribute_mixed の 4 つに限られる。adios2 と hdf5 は連成サブブロックでは現在サポートされていない。

```
# 2次元の流体-固体連成。両側を均質物性とし、
# 減衰 ( $Q_{kappa} = Q_{mu} = 40$ ) を各領域で有効化する例。
material:
  type: coupled

  fluid:
    attribute: 1
    type: isotropic_acoustic
    format: constant
    vp: 1500.0
    rho: 1000.0
    attenuation:
      enabled: true
      f0: 5
      n_units: 3
      Qkappa: 40

  solid:
    attribute: 2
```

```

type: isotropic_elastic
format: constant
vp: 3000.0
vs: 1732.0
rho: 2500.0
attenuation:
  enabled: true
  f0: 5
  n_units: 3
  Qkappa: 40
  Qmu: 40

```

■メッシュ側の要件。 親メッシュは流体要素すべてに `fluid.attribute` の値を、固体要素すべてに `solid.attribute` の値をタグ付けしておく必要がある（Cubit / GMSH で出力される物理グループ ID がそのまま使えることが多い）。加えて、流体要素と固体要素は界面で位相的に接続されている必要がある。Cubit で `subtract` のみを実行して `merge vertex` / `merge curve` を省くと、重複頂点・重複曲線のせいで要素が接続されず、実行時に原因を示すメッセージを出して異常終了する。

4.7.9 モデルエクスポート

キー	型	必須	デフォルト	説明
<code>export_model</code>	bool	任意	false	ADIOS2 BP 形式でモデルをエクスポート
<code>export_dir</code>	string	任意	"./model/"	エクスポート先ディレクトリ

4.7.10 material セクションの設定例

均質弾性体（減衰あり）：

```

material:
  type: isotropic_elastic
  format: constant
  vp: 6000.0
  vs: 3500.0
  rho: 2700.0
  attenuation:
    enabled: true
    f0: 2.0
    n_units: 3
    Qkappa: 100.0
    Qmu: 50.0

```

属性ごとの定義（外部メッシュ使用時）：

```

material:
  type: isotropic_elastic

```

```
format: by_attribute
file: "materials.txt"
attenuation:
  enabled: true
  f0: 2.0
  n_units: 3
```

不均質音響媒質（グリッドファイル）:

```
material:
  type: isotropic_acoustic
  format: grid
  file: "velocity_model.dat"
```

4.8 boundary セクション

4.8.1 吸収境界条件 (absorbing)

Cerjan スポンジ層による吸収境界条件を設定する。

キー	型	必須	デフォルト	説明
type	string	必須	—	現在は cerjan のみ
sides	string 配列	必須	—	適用面（空配列 [] で無効化）
thickness	real	必須	—	スポンジ層の厚さ (m)
alpha	real	必須	—	減衰係数 (4.9)

sides に指定可能な面名:

次元	指定可能な面
2D	bottom, right, top, left
3D	bottom, front, right, back, left, top

■面名と境界属性 ID の対応。 SEMSWS では各面名が固定された 1 始まりの MFEM 境界属性 ID に変換される。この ID は sides でも dirichlet.attributes でも同じ表を用いる。Cubit・GMSH などの外部メッシュジェネレータでメッシュを作成する場合は、境界面にこの ID をそのまま刻印する必要がある。ID がズレると YAML 側の面名が正しい物理境界に対応しない。

表 4.1 面名と SEMSWs が使う MFEM 境界属性 ID の対応。Mesh::MakeCartesian2D・MakeCartesian3D が生成する ID であり、外部メッシュを作る際の面タグ付けでもこの値を踏襲する必要がある。

面名	2D 属性 ID	3D 属性 ID
bottom	1	1 (z 最小)
right	2	3 (x 最大)
top	3	6 (z 最大)
left	4	5 (x 最小)
front	—	2 (y 最小)
back	—	4 (y 最大)

dirichlet.attributes は面名と整数属性 ID の両方を混在して受け付ける。面名は上表の整数 ID に対するエイリアスに過ぎない。

4.8.2 Dirichlet 境界条件 (dirichlet)

キー	型	必須	デフォルト	説明
attributes	配列	任意	[]	面名または整数の境界属性 ID

面名 (top, left 等) と整数属性 ID を混在させることができる。ID の付与には外部メッシュソフト (CUBIT/GMSH など) が必須。内部メッシュではメッシュ領域の境界のみ top, bottom.... など指定可能。SEM では未指定の境界は自然境界条件となるので、音響波動シミュレーションの際は dirichlet が必須となる。

4.8.3 boundary セクションの設定例

```
# 海域シミュレーション：海面を Dirichlet、その他を吸収境界
boundary:
  absorbing:
    type: cerjan
    sides: [bottom, left, right]
    thickness: 1200.0
    alpha: 5.0
  dirichlet:
    attributes: [top]
```

4.9 sources セクション

4.9.1 震源モードと入力形式

キー	型	必須	デフォルト	説明
mode	string	必須	—	simultaneous or sequential
format	string	任意	yaml	震源情報の入力形式: yaml or hdf5
file	string	条件	—	外部震源定義ファイルパス。format=hdf5 時は必須
shot_id	int	任意	0	format=hdf5 時の対象 shot ID (≥ 0)
list	配列	条件	—	インライン震源定義 (format=yaml 時のみ。 format=hdf5 とは排他)

simultaneous は全震源を同時に発火する。sequential は各震源を順番に 1 つずつ処理する (シミュレーション内ループ)。

■format: yaml (既定) . file を指定すると外部 YAML ファイルから list: を読み込む。file を指定しなければインラインの sources.list が使われる。震源数が多い場合や、FWI ワークフローでスクリプトから震源ファイルを自動生成する場合に有用である。

■format: hdf5. SEMSWS HDF5 v2.0 schema から 1 shot 分の震源情報を読み込む経路。詳細スキーマ・制約・使い方は §4.13 にまとめてあるのでそちらを参照。

4.9.2 震源の共通パラメータ (list)

sources.list 配列の各要素に以下を指定する：

キー	型	必須	デフォルト	説明
id	int	必須	—	震源 ID
name	string	任意	—	震源名 (現在のバージョンでは無意味なキー)
type	string	必須	—	force, pressure, moment_tensor
location	real 配列	必須	—	位置 [x,y] or [x,y,z] (m)

4.9.3 震源タイプ: force

弾性波シミュレーション用の single force。

キー	型	必須	説明
direction	real 配列	必須	力の方向ベクトル [x,d] or [x,y,z]

```
- id: 1
  type: force
  location: [500.0, 300.0, 0.0]
  direction: [0.0, 0.0, 1.0]    # 鉛直方向
  wavelet:
    type: ricker
    frequency: 10.0
    amplitude: 1.0e10
```

4.9.4 震源タイプ: pressure

音響波シミュレーション用の圧力点震源。追加パラメータなし。

4.9.5 震源タイプ: moment_tensor

モーメントテンソル震源。地震のメカニズム解を直接指定する。

3次元の場合（6成分）: Mxx, Myy, Mzz, Mxy, Mxz, Myz。

2次元の場合（3成分）: Mxx, Myy, Mxy。

```
- id: 1
  type: moment_tensor
  location: [5000.0, 5000.0, 3000.0]
  moment_tensor:
    Mxx: 1.0e18
    Myy: 0.0
    Mzz: 0.0
    Mxy: 0.0
    Mxz: 0.0
    Myz: 0.0
  wavelet:
    type: ricker
    frequency: 5.0
    amplitude: 1.0
```

4.9.6 ウェーブレット (wavelet)

キー	型	必須	デフォルト	説明
type	string	任意	ricker	ricker, gaussian, external
frequency	real	条件	—	主周波数 (Hz)。ricker/gaussian で必須
amplitude	real	任意	1.0	振幅スケールリング
delay	real	任意	0.0	ピーク到達時刻の遅延 (s)
file	string	条件	—	外部 STF ファイルパス (external 時に必須)

Ricker ウェーブレットでは $\text{delay} \geq 1.2/\text{frequency}$ を推奨する。

外部 STF ファイルは 2 列の ASCII テキスト（時刻, 振幅）である（時刻列は考慮されないのでシミュレーションの開始時刻と 1 行目を合わせる必要がある）：

```
# time (s)      amplitude
0.0000          0.000000e+00
0.0001          1.234568e-05
...
```

4.9.7 観測データ (observed) — インバージョンモード

`simulation.mode` が `inversion` または `misfit_only` の場合、各震源ごとに観測データを `observed:` で指定する。観測データは SEMSWS HDF5 v2.0 schema (`/shots/<NNNN>/receivers/<key>/...`) で書かれた 1 ファイルを参照する。

キー	型	必須	デフォルト	説明
<code>observed.file</code>	string	必須	—	観測データ HDF5 ファイル (.h5) のパス
<code>observed.resample.enabled</code>	bool	任意	false	観測サンプリング間隔を <code>simulation</code> の <code>dt</code> に再標本化するか
<code>observed.resample.method</code>	string	任意	lanczos	再標本化アルゴリズム: lanczos or linear
<code>observed.resample.lanczos_a</code>	int	任意	4	Lanczos カーネルのサポート幅 (method=lanczos 時のみ)

旧 SU schema (`observed.format + observed.files[]`) は廃止された。SU/SEG-Yなどを観測データとして使う場合は事前に SEMSWS HDF5 v2.0 形式へ変換しておく。

4.9.8 sources セクションの設定例

音響波（複数震源、同時発火）：

```
sources:
  mode: simultaneous
  list:
    - id: 1
      type: pressure
      location: [300.0, 400.0]
      wavelet:
        type: ricker
        frequency: 30.0
        amplitude: 1.0e10
        delay: 0.05
```

```
- id: 2
  type: pressure
  location: [700.0, 400.0]
  wavelet:
    type: ricker
    frequency: 30.0
    amplitude: 1.0e10
    delay: 0.05
```

弾性波（モーメントテンソル、外部 STF）：

```
sources:
  mode: sequential
  list:
    - id: 1
      type: moment_tensor
      location: [5000.0, 5000.0, 3000.0]
      moment_tensor:
        Mxx: 1.0e18
        Myy: -1.0e18
        Mzz: 0.0
        Mxy: 0.0
        Mxz: 0.0
        Myz: 0.0
      wavelet:
        type: external
        frequency: 5.0
        file: "stf_earthquake.txt"
```

外部ファイルの構造は config.yaml の sources セクション内の list と同じ形式である：

```
# config.yaml 側
sources:
  mode: simultaneous
  file: "./sources.yaml"

# sources.yaml（外部ファイル）
sources:
  - id: 1
    type: pressure
    location: [1800.0, 4990.0]
    wavelet:
      type: ricker
      frequency: 30.0
      amplitude: 1.0e10
      delay: 0.05
  - id: 2
    type: pressure
    location: [2644.0, 4990.0]
    wavelet:
      type: ricker
```

```
frequency: 30.0
amplitude: 1.0e10
delay: 0.05
```

4.10 receivers セクション

4.10.1 記録タイプと出力設定

キー	型	必須	デフォルト	説明
type	string/配列	必須	—	既定の記録タイプ (受振器ごとに上書き可、後述)
format	string	任意	yaml	受振器情報の入力形式: yaml or hdf5
file	string	条件	—	format=yaml: list:/line: を持つ外部 YAML、format=hdf5: HDF5 ファイル (必須)。§4.10.4 参照。
shot_id	int	任意	0	format=hdf5 時の対象 shot ID (≥ 0)
output.formats	配列	必須	—	出力形式のリスト (マッピング要素)
output.filename	string	条件	—	ベースファイル名 (hdf5/su 時は必須)

■format: hdf5 (受振器位置の入力) . SEMSWS HDF5 v2.0 schema から受振器位置・タイプを読み込む経路。詳細スキーマ・例は §4.13 を参照。出力先 (output.formats, output.filename) は format と独立に YAML で指定する。

type に指定可能な値:

値	説明	対応モデル
DISP	変位	elastic
VEL	速度	elastic
ACC	加速度	elastic
PS	圧力	acoustic

■出力形式。 output.formats はマッピング要素のリストで、各要素の type: キーで ascii (テキスト、受振器ごと 1 ファイル)、hdf5 (単一 HDF5 ファイル)、su (Seismic Unix 形式) のいずれかを選ぶ。複数形式を同時に要求できる:

```
output:
  formats:
    - type: ascii
    - type: su
  filename: "seismograms"      # hdf5 / su を使う場合は必須
```

出力された su ファイルを Seismic Unix で可視化する場合は、XDRFLAG を OFF にしてビルドされた Seismic Unix が必要。

■**受振器ごとのタイプ上書き**。 `list:` の各要素では、親の `type` を上書きする独自の `type:` キー（スカラまたは配列）を指定できる。`material.type: coupled` のシミュレーションで特に有用で、流体側の観測点では [PS] のみ、固体側では [DISP, VEL, ACC] だけを記録するよう分けることができる。これにより、該当サイドで存在しない記録タイプを要求して起きるサブメッシュ側の “receiver type not found” エラーを回避できる。

キー	型	必須	デフォルト	説明
<code>name</code>	string	必須	—	受振器名
<code>location</code>	real 配列	必須	—	位置 [x,y] or [x,y,z] (m)
<code>type</code>	string/配列	任意	親の <code>receivers.type</code>	この受振器にだけ適用する記録タイプ
<code>weight</code>	real	任意	1.0	スカラ重み（FWI 観測データ組立てで使用）

4.10.2 受振器ライン (line)

等間隔に配置された受振器列を自動生成する。

キー	型	必須	デフォルト	説明
<code>line.start</code>	real 配列	必須	—	開始位置 (m)
<code>line.end</code>	real 配列	必須	—	終了位置 (m)
<code>line.count</code>	int	必須	—	受振器数
<code>line.prefix</code>	string	必須	—	名前の接頭辞

名前は `prefix` に 3 桁の番号が付加される（例：`prefix: "REC" → REC001, REC002, ...`）。

4.10.3 receivers セクションの設定例

リスト形式（個別指定）：

```
receivers:
  type: [DISP, VEL]
  output:
    formats:
      - type: ascii
      - type: su
    filename: "seismograms"
  list:
    - name: "R01"
```

```

    location: [1000.0, 2500.0, 0.0]
  - name: "R02"
    location: [2000.0, 2500.0, 0.0]
  - name: "R03"
    location: [3000.0, 2500.0, 0.0]

```

ライン形式（等間隔自動生成）：

```

receivers:
  type: [PS]
  output:
    formats:
      - type: ascii
  line:
    start: [100.0, 300.0]
    end: [900.0, 300.0]
    count: 17
    prefix: "REC"

```

受振器ごとのタイプ上書き例（典型的な流体－固体連成の配置。R001 は流体側、R002/R003 は固体側に置き、それぞれ物理に応じて記録タイプを分ける）：

```

receivers:
  output:
    formats:
      - type: ascii
  type: [PS, DISP, VEL, ACC]    # 下で要求される全タイプの和集合
  list:
    - name: "R001"
      location: [800.0, 200.0]
      type: [PS]
    - name: "R002"
      location: [800.0, 440.0]
      type: [DISP, VEL, ACC]
    - name: "R003"
      location: [800.0, 680.0]
      type: [DISP, VEL, ACC]

```

4.10.4 外部受振器ファイル (receivers.file)

受振器が多い場合は、config.yaml から外部 YAML ファイルへ切り出し、receivers.file キーで参照できる。外部ファイルはトップレベルに list: / line: を持ち、スキーマはインライン指定と全く同じ。type や出力形式、ファイル名はメイン設定側に残しておく。

```

# config.yaml
receivers:
  type: [PS]
  output:
    formats:

```

```
- type: ascii
file: "./receivers.yaml"      # 以下に書いた list/line があっても上書き
```

```
# receivers.yaml - パスはカレントディレクトリから解決される
list:
- name: "REC001"
  location: [100.0, 300.0]
- name: "REC002"
  location: [200.0, 300.0]
- name: "REC003"
  location: [300.0, 300.0]

# line: を list: と併用することも可能
# line:
#   start: [0.0, 300.0]
#   end:   [1000.0, 300.0]
#   count: 21
#   prefix: "LINE"
```

`receivers.file` を指定すると、メイン設定側にインラインで書かれた `list:` / `line:` は無視され、外部ファイルが優先される。外部ファイルが見つからない・YAML 構文エラー等で読み込めない場合は、その旨のメッセージを出して実行を中断する。

4.11 device セクション

キー	型	必須	デフォルト	説明
<code>type</code>	string	任意	cpu	cpu, cuda, hip
<code>seismo_buffer_steps</code>	int	任意	0	receiver 波形バッファサイズ (GPU のみ)

`type` はビルド構成と一致させる必要がある。GPU ビルドで `cpu` を指定すると動作するが非常に低速であるため、CPU 実行には CPU ビルドを使用すべきである。

`seismo_buffer_steps` は GPU 上での波形記録バッファのステップ数を指定する。0 の場合は全ステップを GPU メモリにバッファし（最速だがメモリ消費大）、 $N > 0$ の場合は N ステップごとにホストへ転送する。

4.11.1 device セクションの設定例

```
device:
  type: cuda
  seismo_buffer_steps: 1000
```

4.12 inversion セクション

`simulation.mode` が `inversion` または `misfit_only` のときに任意で配置できるトップレベルセクション。感度カーネルの算出経路を切り替える。

キー	型	必須	デフォルト	説明
<code>inversion.sensitivity.backend</code>	string	任意	<code>hand</code>	感度カーネルのバックエンド: <code>hand</code> (手書きの随伴経路、production) または <code>ad</code> (forward-mode 自動微分による検証用経路)

ad は手書き実装 (hand) と数値的に一致するかを検証するための代替経路で、計算コストは大きい。通常は既定の hand のままで良い。

```
inversion:
  sensitivity:
    backend: hand      # or "ad" for AD verification
```

4.13 HDF5 入出力スキーマと使い方

SEMSWS は震源・受振器・観測データの入力、および受振器波形の出力に YAML とは別に HDF5 v2.0 schema 経由のパスをサポートする。多 shot 構成の FWI ワークフロー (semsws-driver が外側でループ) や、観測データを読み込むインバージョンモードでは、HDF5 が事実上必須の入力形式となる。

スキーマ自体は入出力で共通だが、**入力経路と出力経路**では SEMSWS が「ファイルから何を読むか」「ファイルへ何を書くか」が異なる。本節は両者を明確に分けて説明する。

4.13.1 HDF5 v2.0 スキーマ全体像（共通）

すべての入力 / 出力ファイルはルート属性で `format_version="2.0"` を持ち、`/shots/<NNNN>/` (`<NNNN>` は 4 桁ゼロ詰め shot ID) 配下に 1 shot 分の震源と受振器をまとめる。

[illegible]

/stf	(n_samples,)	f32	# 時系列
STF			
/receivers/<key>/	attrs: position (f32[space_dim]) types (var-length str array; optional) label (str; optional)		
/pressure	(nt,)	f32	# type=PS
の波形			
/velocity	(space_dim, nt)	f32	# type=
VEL の波形			
/displacement	(space_dim, nt)	f32	# type=
DISP の波形			
/acceleration	(space_dim, nt)	f32	# type=
ACC の波形			
/weight_<ch>	same shape as <ch>	f32	# オブ
ション、observed側の重み			

■グループ名 <key> の規約.

- 震源: S<NNNN> (4桁ゼロ詰め)。@id 属性と一致しなければならない (@id=42 なら S0042)。reader はこれを strict に validate する。
- 受振器: 任意の文字列。命名規約なし。@label 属性があればユーザー向け表示名としてそちらを優先、なければグループ名自体が表示名になる。

C++ 側の正準ヘッダは include/srcrcv/HDF5IOSchema.hpp に、Python ドライバ側は driver/src/semsws_driver/io/hdf5_schema.py に attribute / dataset 名定数として一元化され、相互に同期している。

4.13.2 入力経路: HDF5 → SEMSW

震源を HDF5 から渡す (sources.format: hdf5)

```
sources:
  mode: simultaneous          # $\\ref{subsubsec:hdf5-input-shot-loop} 参照
  format: hdf5
  file: inputs/shots.h5
  shot_id: 0                  # /shots/0000/sources/ を読む
```

sources.shot_id で読み込み対象 shot 番号 (4桁ゼロ詰めグループ名にマップ) を指定する。震源の type / position / direction / moment_tensor と STF 時系列がすべて HDF5 から取得されるため、YAML 側の sources.list は記述しない (指定すると validator が排他エラー)。

重要: format: hdf5 経路では STF は時系列データとして必ず /shots/<NNNN>/sources/<key>/stf データセットに格納されていなければならない。wavelet.type: ricker のような YAML による解析ウェーブレットの指定は format: hdf5 と併用できない (YAML からの wavelet は YAML list: 入力経路でのみ機能する)。Ricker などを使いたい場合も、対応する離散時系列を事前に計算して HDF5 に書き込んでおく (semsws-driver 付属ユーティリティで生成可能)。

受振器位置を HDF5 から渡す (receivers.format: hdf5)

```
receivers:
  format: hdf5
  file: inputs/shots.h5
  shot_id: 0                # /shots/0000/receivers/ を読む
  type: [PS]                # ← 全受振器の既定 type (HDF5側で個別上書き可)
  output:
    formats: [{type: ascii}]
    filename: "seis"
```

format=hdf5 を指定した場合、list:/line: は無視される (HDF5 側の受振器集合がそのまま採用される)。

■受振器の記録 type 決定規則 (重要) . 受振器ごとの記録タイプは以下の優先順位で決まる:

1. HDF5 側の受振器グループに @types 属性があればそれを採用 (var-length string array、例: ["DISP", "VEL"])。
2. @types 属性が無い受振器は、YAML 側の receivers.type をデフォルトとして使う。
3. YAML 側 receivers.type も無ければ validator が拒絶する。

例: HDF5 に 3 個の受振器があり、R001 は @types=["PS"]、R002 は @types=["DISP", "VEL"]、R003 は @types なし、YAML の receivers.type: [PS, DISP, ACC] とすると:

- R001 → [PS]
- R002 → [DISP, VEL]
- R003 → [PS, DISP, ACC] (YAML からのデフォルト)

これにより、流体 - 固体連成のシミュレーションで「流体側受振器は PS、固体側受振器は DISP/VEL/ACC」のような分離を HDF5 側に直接書き込めば、YAML 側で全種類の union ([PS, DISP, VEL, ACC]) をデフォルトにしておけば、各受振器に物理的に妥当な type だけが適用される。

観測データ (インバージョン専用)

simulation.mode: inversion または misfit_only では、各震源に対する観測波形を以下のよう指定する:

```
sources:
  list:
    - id: 1
      type: pressure
      location: [...]
      wavelet: { type: ricker, frequency: 5.0, amplitude: 1.0, delay: 0.4 }
      observed:
        file: obs/shot_0000.h5      # 必須
        resample:
          enabled: true
```

```
method: lanczos                      # or "linear"
lanczos_a: 4
```

観測ファイルは §4.13.1 のスキーマに従って `/shots/<NNNN>/receivers/<key>/` 下のデータセット (`pressure`, `velocity`, `displacement`, `acceleration`) を参照する。受振器の `@position` 属性とシミュレーション側の受振器位置が一致しないと `misfit` 計算で誤差が出る。

旧 SU schema (`observed.format + observed.files[]`) は廃止された。SU/SEG-Y などを観測データとして使う場合は事前に SEMSWS HDF5 v2.0 形式へ変換しておく。

shot 間ループと mode の責務分離

HDF5 入力では 1 回の SEMSWS 起動で **1 つの shot だけ** を処理する (`shot_id` で選択)。

- **shot 間ループ** (`/shots/0000`, `/shots/0001`, ... を順に処理) : `semsws-driver` の責務。各 shot ごとに `output.directory` を分離し、独立に並列ジョブとして投入できる。
- **shot 内ループ** (同じ `shot_id` 内に複数 `source = S0001`, `S0042`, ...) : `sources.mode` が制御。
 - `mode: simultaneous` (**推奨**) : shot 内の全 source を同時に発火、1 回のシミュレーション。典型的な FWI の「1 shot = 1 物理イベント」設計と一致。
 - `mode: sequential` : shot 内の source 数 `M` に対して `M` 回シミュレーションを実行し、それぞれの寄与を別ファイルに保存する。研究用 (source 別感度カーネル比較、デバッグ等) に限られる。

入力 HDF5 構造の例

ユースケース別に典型的な入力 HDF5 の中身を 3 例示す。いずれも `root` 属性 (`format_version=2.0`, `dt`, `t0`, `n_samples`, `space_dim`, ...) は省略表記している。

■例 1: 単震源・単 shot (典型的な海底地震計シミュレーション) .

```
inputs/shot.h5
└─ /shots/0000/                                attrs: shot_id=0
   └─ /sources/
      └─ /S0001/                                attrs: id=1, type="pressure",
         │                                     position=[5000, 4000]
         └─ /stf                               (n_samples,) ← 事前計算した時系
            列
      └─ /receivers/
         └─ /R001/  attrs: position=[1000, 0]
            └─ /R002/  attrs: position=[2000, 0]
               └─ /R003/  attrs: position=[3000, 0]
```

対応 YAML:

```
sources:
  mode: simultaneous
  format: hdf5
  file: inputs/shot.h5
  shot_id: 0
```

```

receivers:
  format: hdf5
  file: inputs/shot.h5
  shot_id: 0
  type: [PS]                                # 全受振器に PS を適用

```

■例 2: 複数 shot・shot 内に複数 source (マルチ震源 FWI) .

```

inputs/multishot.h5
├ /shots/0000/                                attrs: shot_id=0
│   ├── /sources/
│   │   ├── /S0001/  attrs: id=1, type="force", direction=[1,0,0]
│   │   │   └ /stf
│   │   └ /S0002/  attrs: id=2, type="moment_tensor",
│   │               moment_tensor=[Mxx,Myy,Mzz,Mxy,Mxz,Myz]
│   │               └ /stf
│   └ /receivers/
│       ├── /STN_001/  attrs: position=[...], types=["DISP","VEL"]
│       └ /STN_002/  attrs: position=[...], types=["VEL"]
├ /shots/0001/                                attrs: shot_id=1
│   ├── /sources/
│   │   └ /S0010/  attrs: id=10, type="pressure"
│   │           └ /stf
│   └ /receivers/
│       ├── /STN_001/  attrs: position=[...], types=["DISP","VEL"]
│       └ /STN_002/  attrs: position=[...], types=["VEL"]
└ /shots/0002/                                attrs: shot_id=2
  ...

```

driver が `shot_id=0,1,2,...` を順次切り替えながら SEMSWS を起動する。各 shot 内で複数 sources が `mode: simultaneous` で同時発火される。受振器ごとの `@types` 属性で、station 別に記録するチャンネルを変えられる。

■例 3: 流体 – 固体連成 (受振器 type を物理に合わせて分ける) .

```

inputs/fluid_solid.h5
├ /shots/0000/                                attrs: shot_id=0
│   ├── /sources/
│   │   └ /S0001/  attrs: id=1, type="pressure", position=[3000, 200]
│   │               (流体側に置いた音響源)
│   │           └ /stf
│   └ /receivers/
│       ├── /OBS_FLUID_01/                    ← 流体側受振器
│       │   attrs: position=[1000, 100], types=["PS"]
│       ├── /OBS_FLUID_02/
│       │   attrs: position=[2000, 100], types=["PS"]
│       ├── /OBS_SOLID_01/                    ← 固体側受振器
│       │   attrs: position=[1500, 800], types=["DISP", "VEL"]
│       └ /OBS_SOLID_02/
│           attrs: position=[2500, 800], types=["DISP", "VEL"]

```

対応 YAML (ここでは `receivers.type` は union を指定して、HDF5 側 `@types` による上書きで物理整合性を取る) :

```
receivers:
  format: hdf5
  file: inputs/fluid_solid.h5
  shot_id: 0
  type: [PS, DISP, VEL]      # union; 各受振器の @types で上書きされる
```

4.13.3 出力経路: SEMSWS → HDF5

受振器波形の HDF5 出力 (output.formats: hdf5)

receivers.output.formats に type: hdf5 を含めると、シミュレーション結果の受振器波形が §4.13.1 のスキーマで HDF5 に保存される。output.filename (.h5 拡張子は省略可) がベース名となり、後述の source ID suffix が付く。

```
receivers:
  type: [PS]
  output:
    formats: [{type: hdf5}]
    filename: "seis"          # → seis<NNNN>.h5
  list:
    - { name: "R001", location: [1000, 500] }
    - { name: "R002", location: [2000, 500] }
```

(ascii や su 等の他形式と併用可、それらの命名・構造は §4.10.1 を参照。本節以降は HDF5 に話を絞る。)

出力ファイル中の受振器グループ名(<key>)には、YAML の list[] .name (または line.prefix + 3 桁番号) がそのままグループ名として使われる (reader 側と異なり writer 側は受振器名の変換を行わない)。例えば上記の例では seis<NNNN>.h5:/shots/0000/receivers/R001/ のように出力される。

震源グループ側は S<NNNN>規約に従って自動命名される。

出力ファイル名と内部 shot ID の対応

SEMSWS の 1 回の起動で生成される受振器出力ファイルの命名と内部/shots/<NNNN>/キーは、mode で挙動が分かれる。

■mode: simultaneous (推奨) . shot 内の source 数によらず各出力形式につき 1 セットのみ生成される (全 source が合成された波形を 1 ファイルに保存)。ファイル名の suffix (4 桁ゼロ詰め) と内部/shots/<NNNN>/キーは両方とも入力 sources.shot_id を採用する。

例: 入力 sources.shot_id=42、shot 42 内に source S0001, S0042, S0100 があり、3 つを同時発火するケース:

```
out/seis0042.h5          # 1 ファイルだけ、suffix=入力 shot_id=42
```

seis0042.h5 の中身 (self-roundtrip 仕様、§4.13.3) :

```
seis0042.h5
└ root attrs: format_version=2.0, dt, t0, n_samples, ...
```

```

└ /shots/0042/                                attrs: shot_id=42 ← 入力と一致
  └ /sources/                                  ← 全 active source のメタデータ
    └ /S0001/ (id=1, type, position, /stf, ...)
    └ /S0042/ (id=42, ...)
    └ /S0100/ (id=100, ...)
  └ /receivers/                                ← 全受振器の波形(合成)
    └ /R001/ (position, /pressure, /velocity, ...)

```

入力 `shot_id` が出力ファイル名と内部キー両方に反映されるため、ファイル単独で「これは入力 `shot` 何番の結果か」が自明にわかる。

■`mode: sequential`. `shot` 内の `source` 数 `M` に対して `M` 回シミュレーションが走り、各形式につき `M` 個のファイルが生成される。各ファイルはその `iteration` で `active` だった 1 つの `source` が単独発火した時の出力。ファイル名 `suffix` は各 `iteration` の `source.@id` を採用する (`M` 個のファイルを区別するため)。内部 `/shots/<NNNN>/` キーは入力 `shot_id` (全 `iteration` で共通)。

例: 入力 `shot_id=42`、`shot` 内 `source` `S0001`, `S0005`, `S0009`:

```
out/seis0001.h5, out/seis0005.h5, out/seis0009.h5 # HDF5 × 3
```

各 HDF5 ファイルの中身は **1 source 単独発火の結果のみ**を含む (合成ではない) :

```

seis0001.h5                                ← S0001 単独で発火した時の波形
└ root attrs: ...
  └ /shots/0042/                                attrs: shot_id=42 ← 入力 shot_id と一致
    └ /sources/
      └ /S0001/                                ← active 1個だけ
    └ /receivers/
      └ /R001/ (...)

seis0005.h5                                ← S0005 単独で発火した時の波形
└ /shots/0042/                                attrs: shot_id=42 ← 同じ shot 内の別
  iteration
  └ /sources/
    └ /S0005/                                ← active 1個だけ
  └ /receivers/
    └ /R001/ (...)

seis0009.h5                                ← 同様
└ /shots/0042/
  └ /sources/
    └ /S0009/
  └ /receivers/
    └ /R001/ (...)

```

3 個の HDF5 すべて内部 `/shots/0042/` (入力 `shot_id` と一致)。ファイル間の区別は**ファイル名の suffix** (`source.id`: 0001, 0005, 0009) でつける。

Self-roundtrip 仕様 と driver merge

■Self-roundtrip. 出力された各 `seis<NNNN>.h5` は `/sources/` と `/receivers/` を両方含むため、そのまま SEMSWs の入力として再利用できる。例えば forward 結果を観測データとして次のインバージョンに渡す、forward 再現テストの参照ファイルとして使う、等。

■入力 shot との対応の取り方. 内部 `/shots/<NNNN>/` キーと `@shot_id` 属性は、入力 `sources.shot_id` と一致するように書き出される。さらに simultaneous モードではファイル名 suffix も `shot_id` に一致するため、出力ファイル単独で「これは入力 shot 何番の結果か」が自明。

`semsws-driver` を使う場合、driver が自動で per-shot directory layout を構築する:

```
output_root/
├ shot_0000/                                ← 入力shot 0 の処理結果
│   ├── config.yaml                        (shot 0 用に書き換えた YAML、shot_id=0)
│   ├── seismograms0000.h5                (内部 /shots/0000/、suffix も 0000)
│   └── stdout.log
├ shot_0001/                                ← 入力shot 1
│   ├── config.yaml                        (shot_id=1)
│   ├── seismograms0001.h5                (内部 /shots/0001/、suffix 0001)
│   └── stdout.log
└ shot_0002/
    ├── config.yaml                        (shot_id=2)
    ├── seismograms0002.h5                (内部 /shots/0002/、suffix 0002)
    └── stdout.log
```

`driver/src/semsws_driver/core/layout.py` の `shot_dir(shot_id)` で directory 命名され、driver の manifest にも記録される。driver は per-shot YAML 生成時に `sources.shot_id` と `receivers.shot_id` を入力 shot に合わせ、`receivers.output.filename` を "seismograms" (固定) に設定する (`driver/src/semsws_driver/core/template.py`)。結果として directory 名 `shot_<NNNN>` とファイル名 suffix `<NNNN>`、ファイル内部 `/shots/<NNNN>/` の三重で shot identity が表現される。

driver を使わず手で SEMSWs を回す場合も、各 invocation で `sources.shot_id` を変えるだけで自動的にファイル名と内部キーが分離するため衝突しない:

```
for SHOT in 0 1 2; do
  yq -i ".sources.shot_id = $SHOT |
        .simulation.output.directory = \"./out/shot_$(printf %04d $SHOT)\"\"
        \
        config.yaml
  mpirun -np 8 semsws --config config.yaml
done
```

各 shot で `seis0000.h5`, `seis0001.h5`, `seis0002.h5` がそれぞれ別 directory に生成される。`output.directory` を分離しなくてもファイル名 suffix で区別されるが、log や config が shot 間で混ざるので directory 分離が推奨。

■driver の merge ツール. `semsws-driver` が複数 shot を順に走らせた場合、各 shot から生成される per-shot ファイル群を 1 つの HDF5 にまとめる post-process が用意されている (driver/tools/merge_shot_outputs.py 相当)。出力例:

```
shots_merged.h5
└─ /shots/
   ├── /0000/ (shot 0 の sources + receivers)
   ├── /0001/ (shot 1 の ...)
   └── /0002/ (...)
```

各 per-shot ファイル中の `/shots/<NNNN>/` がそのまま正しい index に入るため、merge ツールは `shot_id` の振り直しが不要で、単純に連結するだけで済む。

出力構成の例 (YAML 設定 → 生成 HDF5)

■例 1: 単震源・simultaneous (典型的な forward シミュレーション) . YAML:

```
sources:
  mode: simultaneous
  shot_id: 0 # 省略すると 0
  list:
    - id: 1
      type: pressure
      location: [5000, 4000]
      wavelet: { type: ricker, frequency: 5.0, amplitude: 1.0, delay: 0.4 }
receivers:
  type: [PS]
  output:
    formats: [{type: hdf5}]
    filename: "seis"
  list:
    - { name: "R001", location: [1000, 0] }
    - { name: "R002", location: [2000, 0] }
```

生成ファイル (1 個) :

```
out/seis0000.h5 ← suffix=入力 shot_id=0
└─ /shots/0000/   attrs: shot_id=0
   ├── /sources/
   │   ├── /S0001/ attrs: id=1, type="pressure", position=[5000,4000]
   │   │   └── /stf (n_samples,)
   │   └── /receivers/
   │       ├── /R001/ attrs: position=[1000,0]
   │       │   └── /pressure (nt,)
   │       └── /R002/ attrs: position=[2000,0]
   │           └── /pressure (nt,)
```

■例 2: 複数震源・simultaneous (同時発火、合成波形が 1 ファイル) . YAML:

```
sources:
```

```

mode: simultaneous          # ← 全 source 同時発火
shot_id: 7                  # ← 入力 shot_id=7 と仮定
list:
  - id: 1; type: force; location: [...]; direction: [1,0]; wavelet: {...}
  - id: 2; type: force; location: [...]; direction: [0,1]; wavelet: {...}
  - id: 3; type: pressure; location: [...]; wavelet: {...}
receivers:
  type: [DISP, VEL]
  output:
    formats: [{type: hdf5}]
    filename: "seis"
  list:
    - { name: "STN_001", location: [...] }

```

生成ファイル (HDF5 1 個に 3 source 合成結果が入る) :

```

out/seis0007.h5          ← suffix=入力 shot_id=7
└ /shots/0007/          attrs: shot_id=7
  └ /sources/            ← 全 active source のメタデータ
    └ /S0001/ (force, /stf)
    └ /S0002/ (force, /stf)
    └ /S0003/ (pressure, /stf)
  └ /receivers/
    └ /STN_001/          ← 3 source 同時発火の合成波形
      └ /displacement    (space_dim, nt)
      └ /velocity        (space_dim, nt)

```

重要: 3 source あっても HDF5 ファイルは 1 個。各受振器の波形データセットには 3 source 合成結果が入る。ファイル名 suffix・内部/shots/<NNNN>/ 共に入力 shot_id を反映。

■例 3: 複数震源・sequential (source 別個出力) . YAML:

```

sources:
  mode: sequential          # ← M iteration、各 source を別個に発火
  shot_id: 42              # ← 入力 shot_id=42 と仮定
  list:
    - id: 1; type: pressure; location: [...]; wavelet: {...}
    - id: 5; type: pressure; location: [...]; wavelet: {...}
    - id: 9; type: pressure; location: [...]; wavelet: {...}
receivers:
  type: [PS]
  output:
    formats: [{type: hdf5}]
    filename: "seis"
  list:
    - { name: "R001", location: [...] }

```

生成ファイル (HDF5 は 3 個、各 source 単独発火の結果。suffix は sequential では各 iteration の source.id) :

out/seis0001.h5	← S0001 単独発火、suffix=source.id
=1	
└ /shots/0042/	attrs: shot_id=42 (入力と一致)
└ /sources/	
└ /S0001/ (id=1, /stf)	← active 1個だけ
└ /receivers/	
└ /R001/	
└ /pressure	(nt,) ← S0001 のみの寄与
out/seis0005.h5	← S0005 単独発火
└ /shots/0042/	attrs: shot_id=42 (同じ shot 内)
└ /sources/	
└ /S0005/ (id=5, /stf)	
└ /receivers/	
└ /R001/	
└ /pressure	← S0005 のみの寄与
out/seis0009.h5	← S0009 単独発火
└ /shots/0042/	attrs: shot_id=42
└ /sources/	
└ /S0009/ (id=9, /stf)	
└ /receivers/	
└ /R001/	
└ /pressure	← S0009 のみの寄与

3 個の HDF5 すべて内部 /shots/0042/ (入力 shot_id と一致)。ファイル間の区別は**ファイル名 suffix の source.id**。各ファイル単独で self-roundtrip 可能 (次の inversion 等の入力に再利用可)。

第 5 章

ツールとスクリプト

SEMSWS にはシミュレーション本体 (semsws) に加え、メッシュの分割・細分化・品質評価などの前処理ツールと、結果の可視化用 Python スクリプトが付属する。本章ではこれらの使い方を説明する。

5.1 partition_mesh — メッシュの事前分割

partition_mesh は、メッシュファイルを MPI 並列実行用に事前分割するツールである (serial run only)。分割済みメッシュを使うことで、シミュレーション起動時のメッシュ読み込み・分割のオーバーヘッドを省略する。同じメッシュ、分割数でシミュレーションを複数行うときに便利。

5.1.1 使い方

```
# 外部メッシュを METIS で 64 分割
./build/src/partition_mesh -n 64 domain.exo partitions/

# Cartesian 分割 (4*4*4 = 64)
./build/src/partition_mesh -n 64 -p cartesian -g 4,4,4 domain.exo partitions
/

# 内部メッシュを生成して分割
./build/src/partition_mesh -n 64 --internal \
  --size 10000,10000,5000 --elements 100,100,50 partitions/
```

5.1.2 オプション

オプション	説明
<code>-n <nparts></code>	分割数（必須）
<code>-p <method></code>	分割方法： <code>metis</code> （デフォルト）または <code>cartesian</code>
<code>-g <nx,ny,nz></code>	Cartesian 分割のグリッド数
<code>--internal</code>	内部メッシュ生成モード
<code>--dim <2 3></code>	次元（内部メッシュ時、デフォルト：3）
<code>--origin <x,y,z></code>	原点（内部メッシュ時）
<code>--size <lx,ly,lz></code>	ドメインサイズ（内部メッシュ時、必須）
<code>--elements <nx,ny,nz></code>	要素数（内部メッシュ時、必須）

出力ファイルは指定ディレクトリに `mesh.mesh.000000`, `mesh.mesh.000001`, ... として生成される。

5.1.3 config.yaml での使用

分割済みメッシュをシミュレーションで使用するには、`config.yaml` の `mesh` セクションを以下のように設定する：

```
mesh:
  type: partitioned
  partitioned:
    directory: "partitions/"
    nparts: 64
```

`nparts` は MPI 実行時のプロセス数と一致させる必要がある。

5.2 combine_mesh — 分割メッシュの結合

`combine_mesh` は、`partition_mesh` で分割したメッシュを 1 つのファイルに結合するツールである。GLVis での可視化や、分割メッシュの確認に使用する。

```
# 64分割のメッシュを結合（分割数と同じnpで実行）
mpirun -np 64 ./build/src/combine_mesh partitions/ combined.mesh
```

実行時の MPI プロセス数は分割数と一致させる必要がある。

結合されたメッシュは GLVis で確認できる：

```
glvis -m combined.mesh
```

5.3 evaluate_mesh — メッシュ品質の評価

evaluate_mesh は、メッシュの品質を評価し、シミュレーションパラメータとの整合性を確認するツールである。要素サイズ、CFL 条件、PPW (Points Per Wavelength)、アスペクト比、ヤコビアン行列式の統計量を算出する。

5.3.1 使い方

```
# 基本的な評価（コンソール出力のみ）
mpirun -np 4 ./build/src/evaluate_mesh --config config.yaml

# ヒストグラム CSV ファイルも出力
mpirun -np 4 ./build/src/evaluate_mesh --config config.yaml \
  --histogram mesh_hist --nbins 50
```

5.3.2 オプション

オプション	説明
--config <file>	設定ファイル（必須）
--cfl <factor>	CFL 係数のオーバーライド
--histogram <prefix>	ヒストグラム CSV の出力接頭辞
--nbins <N>	ヒストグラムのビン数（デフォルト：50）

5.3.3 出力されるヒストグラム

--histogram を指定すると、以下の CSV ファイルが生成される：

ファイル	内容
<prefix>_element_size.csv	要素サイズ分布
<prefix>_dt_cfl.csv	CFL 条件による $\Delta t_{\max}$ 分布
<prefix>_ppw.csv	PPW 分布
<prefix>_aspect_ratio.csv	アスペクト比分布
<prefix>_jacobian_det.csv	ヤコビアン行列式分布

5.4 refine_mesh — メッシュの均一細分化と Partitioning

refine_mesh は、物性値に基づいてメッシュを均一に細分化するツールである。低速度域（波長が短い領域）の要素を自動的に分割し、PPW 条件を満たすメッシュを生成する。

細分化の基準は以下の式で決まる。まず、必要な要素サイズ h_{required} を求める：

$$h_{\text{required}} = \frac{v_{\min} \times N_{\text{GLL}}}{f_{\max} \times \text{PPW}}$$

ここで $v_{\min}$ は全要素中の最小波速度、 $N_{\text{GLL}} = \text{order} + 1$ 、 $f_{\max}$ と PPW は config の `mesh.max_freq` と `mesh.ppw` で指定する。

次に、現在の要素サイズ h_{current} から細分化回数 n を決定する：

$$n = \left\lceil \log_2 \frac{h_{\text{current}}}{h_{\text{required}}} \right\rceil$$

$h_{\text{current}} \leq h_{\text{required}}$ であれば細分化は不要 ($n = 0$)。全要素の最大 n がメッシュ全体の細分化レベルとなる。

注意として、メッシュ全体を均一に細分化する。1 回の細分化で各要素が 2 分割されるため、2 次元では要素数が 2^{2n} 倍、3 次元では 2^{3n} 倍になる。なので、PPW などの指標はある程度妥協が必要な時がある。

使用用途としては、PPW を満たすメッシュの作成がまず一つ。二つ目に HPC での大規模計算（大規模メッシュ）における用途がある。**SEMSWS では並列計算であっても (`mesh.type.partitioned` in `config.yaml` 以外のとき)、1 rank で mesh 全体を読み込んでその後分割し、MPI_SEND を行う。そのため、大規模な mesh を扱う際、メモリに載らない問題が発生する。また、CUBIT などで事前に `partitioning` されたメッシュも読み込むことができない。`partition_mesh` の事前分割も最初は 1rank で全 mesh を読み込んでしまう。** こういうときは、ある程度の geometry (topography や bathymetry) を捉えることのできる粗い mesh を CUBIT や GMSH で作成し、refine で細分化する方法がある。refine は次に説明するように、入力された mesh を 1 rank で読み込む → mesh 分割 → 各ランクで refine → partitioned mesh 出力を行う。出力された mesh は `mesh.type.partitioned` を config に設定することで各ランクがそれぞれのパートの mesh のみを読み込む。

5.4.1 メッシュの読み込みと出力の挙動

`refine_mesh` は内部で `LoadParMesh()` を呼び出してメッシュを読み込む。読み込みパスは config の `mesh.type` によって異なる：

■未分割メッシュ (type: internal / external) の場合 Rank 0 のみがシリアルメッシュを読み込み (internal の場合は生成し)、`MeshPartitioner` で N 分割した後、各ランクに `MPI_Send` で配布する。 N は `mpirun -np N` のプロセス数で決まる。

■分割済みメッシュ (type: partitioned) の場合 各ランクが自分のパーティションファイル (`mesh.mesh.NNNNNN`) を独立に読み込む。実行時の MPI プロセス数と `nparts` が一致している必要がある。

■リファインと出力 どちらのケースでもリファイン自体は `ParMesh::UniformRefinement()` で並列に動作し、挙動に違いはない。違うのは読み込みパスのみである。リファイン後の出力は 2 通り：

- `--output-dir`: 各ランクが `ParPrint()` で自分のパーティションファイルを書き出す (N 個のファイル)。シミュレーションで type: partitioned として使用する

- `--save-as-one`: Rank 0 が `SaveAsOne()` で 1 つのファイルに結合する。GLVis での可視化やシミュレーションで `type: external` として使用する

5.4.2 使い方

```
# 8プロセスで実行 (MPI必須)
mpirun -np 8 ./build/src/refine_mesh --config config.yaml --output-dir MESH/

# 可視化用の結合メッシュも同時に保存
mpirun -np 8 ./build/src/refine_mesh --config config.yaml \
  --output-dir MESH/ --save-as-one refined.mesh
```

5.4.3 オプション

オプション	説明
<code>--config <file></code>	設定ファイル (必須)
<code>--output-dir <dir></code>	分割メッシュの出力先 (必須)
<code>-n <max_ref></code>	最大細分化レベルの上書き
<code>--save-as-one <file></code>	結合メッシュも保存 (GLVis 可視化用)

出力は `partition_mesh` と同じ形式 (`mesh.mesh.NNNNNN`) であり、`config.yaml` で `type: partitioned` として直接使用できる。

5.5 ワークフロー例：メッシュ評価と細分化

ここでは `examples/visco_elastic_3d_layered/` の 2 層粘弾性モデルを例に、メッシュの評価から細分化、再評価までの一連のワークフローを示す。

5.5.1 問題設定

10 km × 10 km × 10 km の 3 次元ドメインに 2 層の粘弾性媒質を設定する。上層 ($z > -4000$ m) は低速度 ($V_p = 3000$ m/s, $Q_\kappa = Q_\mu = 50$)、下層 ($z \leq -4000$ m) は高速度 ($V_p = 6000$ m/s, $Q_\kappa = Q_\mu = 100$) である。

メッシュは CUBIT (無料版 Coreform で可) で生成する。付属のジャーナルファイル `mesh_layered_3d.jou` を実行すると ExodusII 形式のメッシュが生成される：

```
# CUBIT GUI CONSOLEで以下を実行
playback 'mesh_layered_3d.jou'
```

物性ファイル `materials_visco.txt` の内容：

```
# attribute  vp      vs      rho      qkappa  qmu
```

1	6000.0	3500.0	2700.0	100.0	100.0
2	3000.0	2000.0	2200.0	50.0	50.0

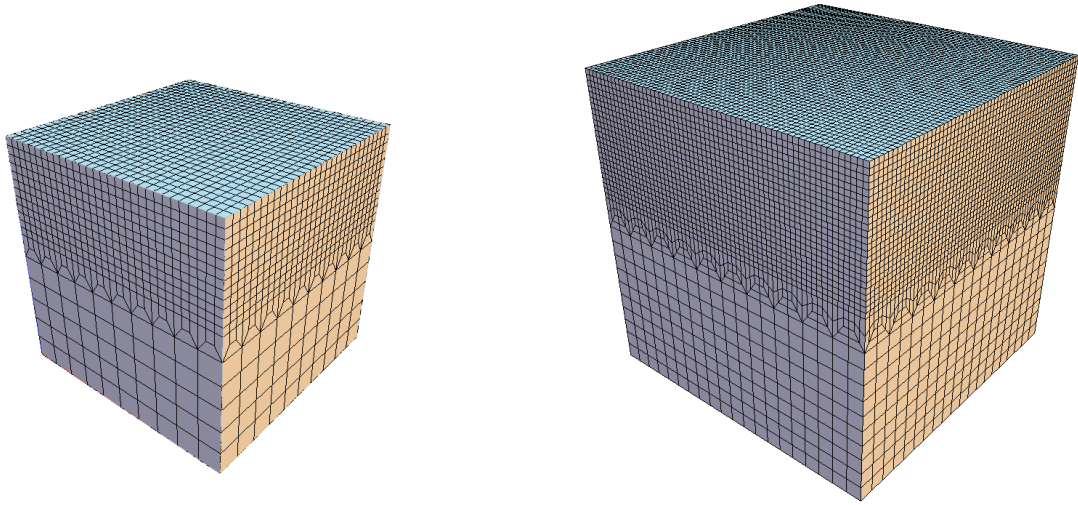


図 5.1 細分化前 (左) と細分化後 (右) のメッシュ。左: CUBIT で生成した初期メッシュ (12,600 要素)。上層は 500 m、下層は 1,000 m 要素。右: `refine_mesh` で 1 回均一細分化後 (100,800 要素)。全要素が $2^3 = 8$ 倍に分割されている。

5.5.2 Step 1: 初期メッシュの評価

まず初期メッシュの品質を評価する:

```
./build/src/evaluate_mesh --config config_visco.yaml --histogram mesh
```

出力例:

```
Mesh Evaluation Report
=====
Mesh: 12600 elements (global), dimension 3
Order: 4, CFL factor: 0.3, f_max: 5.6 Hz, PPW target: 5

Per-element statistics:

```

	min	max	mean	std
h [m]	3.333e+02	1.000e+03	3.858e+02	1.561e+02
dt_CFL [s]	2.141e-03	8.562e-03	5.394e-03	1.227e-03
PPW	3.163e+00	9.490e+00	5.436e+00	7.247e-01
Aspect ratio	1.000e+00	8.492e+00	1.331e+00	1.191e+00
det(J)	2.778e+07	1.000e+09	7.540e+07	1.883e+08

```

CFL summary:
  dt_max (global): 2.1406e-03 s

PPW summary:
  Worst PPW: 3.16 at centroid (9500.00, 4500.00, -6500.00)
  PPW < 5.0: 1300 / 12600 (10.32%)

```


Per-attribute breakdown:

```
Attr 1: 1800 elems, Vp=6050, Vs=3543, PPW_min=3.2, dt_min=2.14e-03
Attr 2: 10800 elems, Vp=3066, Vs=2049, PPW_min=5.5, dt_min=5.63e-03
```

PPW の最小値が 3.16 と目標の 5 を下回っており、Attr 1 (下層、高速度域) の 1300 要素 (10.3%) で PPW 不足であることがわかる。

5.5.3 Step 2: 均一細分化

`refine_mesh` でメッシュを細分化する。ここでは 4 プロセスで実行するが、付属の `run_visco.sh` では 64 プロセスで実行している：

```
mpirun -np 4 ./build/src/refine_mesh \
  --config config_visco.yaml \
  --output-dir mesh/
```

出力例：

```
Loading mesh...
  Elements (global): 12600

Loading material...
  V_min (global): 2049.22
  V_max (global): 6050.01

Refinement levels needed: 1
  Refinement 1: 100800 elements (global)

Refined mesh:
  Elements (global): 100800
  h range:           [84.12, 500.03]
  PPW (worst):       6.33 (target: 5)
  CFL dt_max:        0.00107 s (CFL=0.3)

Saving partitioned mesh to: mesh/
  4 partition files (mesh.mesh.NNNNNN)
```

1 回の細分化で 12,600 要素が 8 倍の 100,800 要素となり、PPW 最小値が 3.16 から 6.33 に改善された (図 5.1)。

5.5.4 Step 3: 細分化後の再評価

細分化後のメッシュを再評価し、PPW 条件が満たされたことを確認する。`config_visco_sim.yaml` では `mesh.type: partitioned` で細分化済みメッシュを参照する：

```
mpirun -np 4 ./build/src/evaluate_mesh --config config_visco_sim.yaml
```

出力例：

```

Mesh Evaluation Report
=====
Mesh: 100800 elements (global), dimension 3
Order: 4, CFL factor: 0.3, f_max: 5.6 Hz, PPW target: 5

Per-element statistics:

              min              max              mean              std
h [m]          1.667e+02      5.000e+02      1.938e+02      8.145e+01
dt_CFL [s]      1.070e-03      4.281e-03      2.718e-03      5.622e-04
PPW             6.326e+00      1.898e+01      1.087e+01      1.563e+00
Aspect ratio    1.000e+00      8.495e+00      1.230e+00      8.541e-01

PPW summary:
Worst PPW: 6.33 at centroid (250.00, 9250.00, -9250.00)
PPW < 5.0: 0 / 100800 (0.00%)

Per-attribute breakdown:
Attr 1: 14400 elems, Vp=6050, Vs=3543, PPW_min=6.3, dt_min=1.07e-03
Attr 2: 86400 elems, Vp=3066, Vs=2049, PPW_min=11.0, dt_min=2.82e-03

```

PPW 不足の要素が 0% となり、全要素で $PPW \geq 5$ の条件を満たしている。

5.5.5 Step 4: シミュレーションの実行

細分化済みメッシュでシミュレーションを実行する：

```

mpirun -np 64 ./build/src/semsws \
  --config config_visco_sim.yaml

```

5.6 可視化スクリプト

scripts/plot/に Python による可視化スクリプトが用意されている。いずれも matplotlib と numpy が必要である。

5.6.1 plot_wavefield_2d.py — 2D 波動場の可視化

GMT 形式で出力された 2D 波動場スナップショットと物性分布を matplotlib で可視化する。

```

python3 scripts/plot/plot_wavefield_2d.py --results-dir ./results

```

results/figures/に以下のファイルが生成される：

- wavefield_snapshots.png — 波動場スナップショット（全タイムステップ）
- material.png — 物性分布

出力例は第 3 章の図 3.1 を参照。

5.6.2 plot_wavefield_3d.py — 3D 断面波動場の可視化

GMT 形式の 3D 断面 (cross_sections) を matplotlib で可視化する。

```
python3 scripts/plot/plot_wavefield_3d.py --results-dir ./results
```

config で指定した各断面 (yz, xz, xy) ごとにスナップショットが生成される。図 5.2 に 2 層粘弾性モデルの xz 断面 ($y = 5000$ m) の出力例を示す。

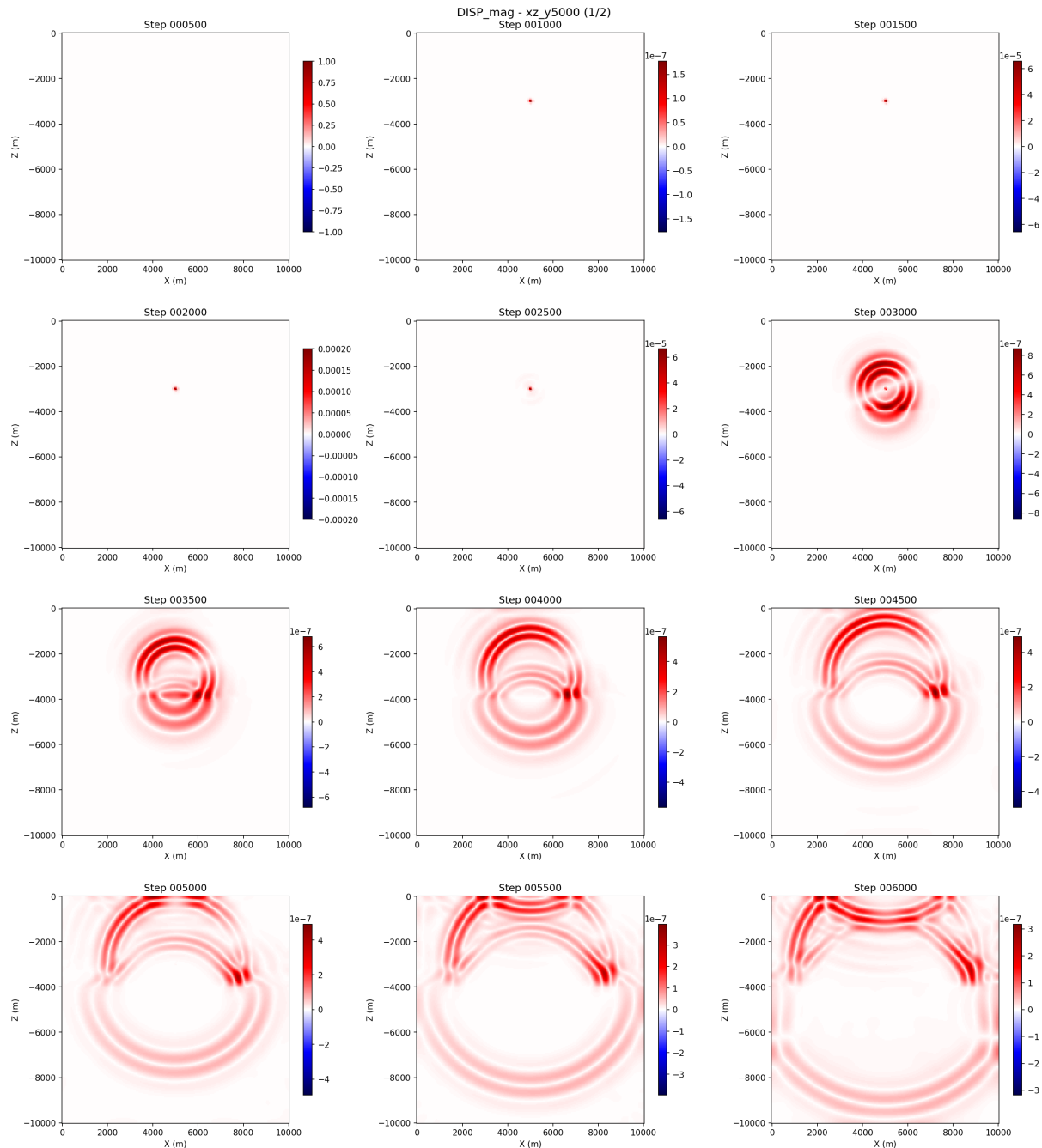


図 5.2 plot_wavefield_3d.py による 3D 断面スナップショット (xz 断面、 $y = 5000$ m)。2 層粘弾性モデルでの変位マグニチュード。500 ステップ間隔で出力。

5.6.3 plot_3d_slices.py — PyVista 3D 可視化

PyVista を使用して、3つの断面を 3D 空間上に配置した可視化を行う。pyvista パッケージが追加が必要である。

```
python3 scripts/plot/plot_3d_slices.py --results-dir ./results \
  --type wavefield --step 6000
```

オプション	説明
--results-dir	結果ディレクトリ
--type	wavefield or material
--step	表示するタイムステップ番号 (wavefield 時)

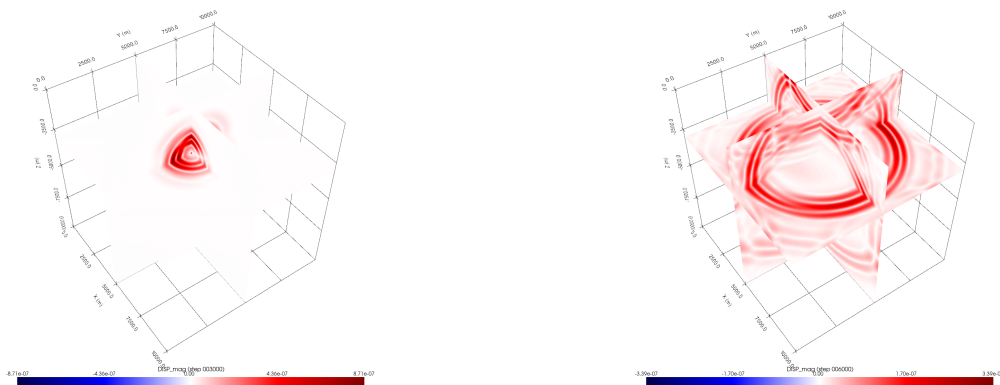


図 5.3 plot_3d_slices.py による PyVista 3D 可視化。yz, xz, xy 断面を 3D 空間上に配置。左：Step 3000、右：Step 6000。2 層粘弾性モデルの変位マグニチュード。

5.6.4 plot_mesh_quality.py — メッシュ品質ヒストグラム

evaluate_mesh が出力した CSV ファイルからヒストグラム図を生成する。

```
# evaluate_meshでCSV出力
mpirun -np 4 ./build/src/evaluate_mesh --config config.yaml \
  --histogram mesh_hist

# ヒストグラム図を生成
python3 scripts/plot/plot_mesh_quality.py mesh_hist
python3 scripts/plot/plot_mesh_quality.py mesh_hist --output quality.png
```

要素サイズ、CFL $\Delta t_{\max}$ 、PPW、アスペクト比、ヤコビアン行列式の各ヒストグラムが 1 枚の図にまとめられる。

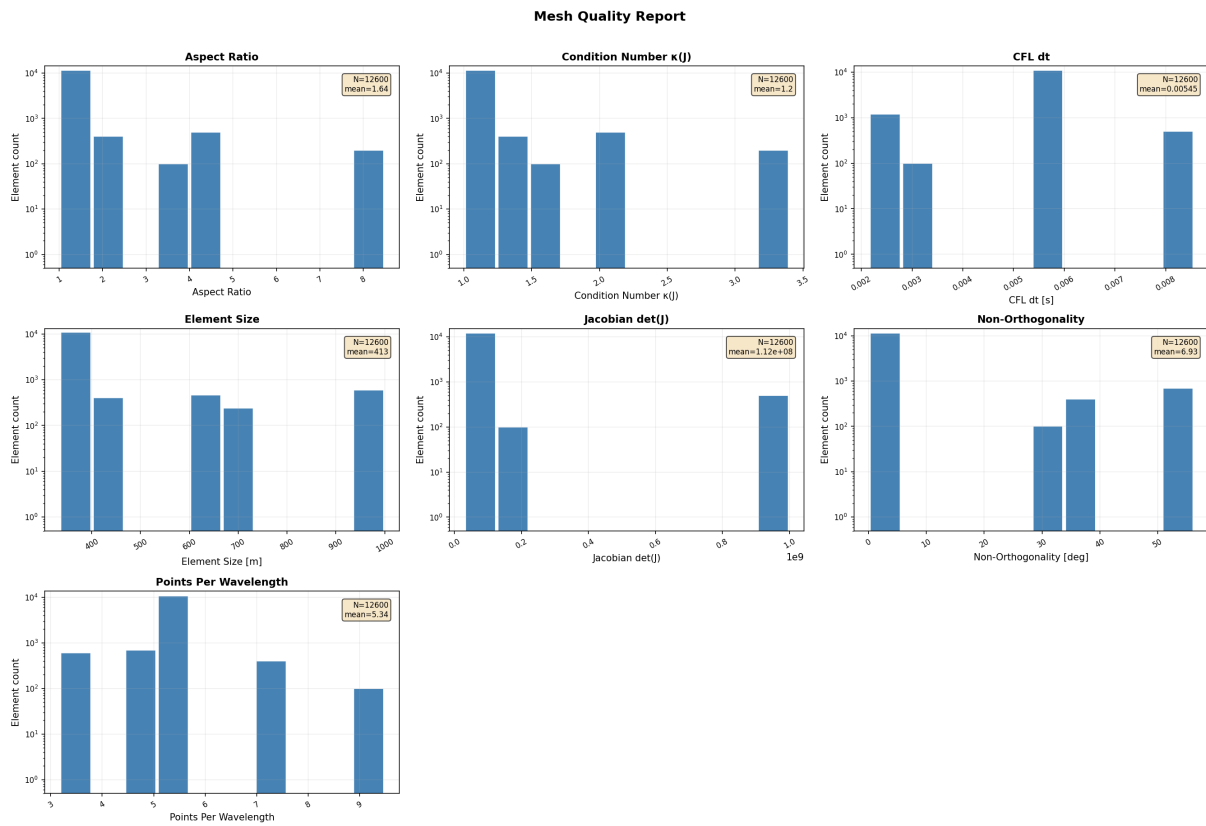


図 5.4 メッシュ品質ヒストグラム（細分化前の初期メッシュ）。`evaluate_mesh` の出力を `plot_mesh_quality.py` で可視化。

第 6 章

応用例

examples/ディレクトリには、SEMSWS の各機能を示すサンプルが用意されている。本章では FWI 関連を除く主要な例題を紹介する。各例題には README と実行スクリプトが含まれており、手順に従って実行できる。

6.1 visualization — 可視化デモ

examples/visualization/には、2D/3D の音響波・弾性波シミュレーションですべての出力フォーマット（GLVis, ParaView, GMT）をテストするデモが含まれている。

ディレクトリ	モデル	説明
2D/acoustic/	2D 音響波	第 3 章で使用
2D/elastic/	2D 弾性波	force 震源
3D/acoustic/	3D 音響波	3D 断面出力 (cross_sections)
3D/elastic/	3D 弾性波	モーメントテンソル震源

各例題は run.sh で実行できる：

```
cd examples/visualization/3D/elastic
bash run.sh 4      # 引数は MPI プロセス数
```

run.sh はシミュレーション実行後、plot_wavefield_2d.py (2D) または plot_wavefield_3d.py (3D) で自動的に可視化を行う。

3D 例題では config の cross_sections で断面位置を指定しており、第 5 章の図 5.2 および図 5.3 はこの 3D 弾性波例題と同様の設定で生成したものである。

6.2 elastic_analytic — 解析解ベンチマーク

examples/elastic_analytic/は、均質等方弾性体中のダブルカップル震源に対する Aki & Richards (2002) の解析解と数値解を比較するベンチマークである。

6.2.1 問題設定

パラメータ	値
ドメイン	44 km × 44 km × 34 km
メッシュ	内部生成、120 × 120 × 100 要素
多項式次数	4
物性	$V_p = 2800 \text{ m/s}$, $V_s = 1500 \text{ m/s}$, $\rho = 2300 \text{ kg/m}^3$
震源	ダブルカップル (M_{rp} 成分)、ドメイン中心
受振器	6 点 (Z1-Z6)、震源からの距離を変えて配置

6.2.2 実行方法

```
cd examples/elastic_analytic
bash run_benchmark.sh 4      # 引数は MPI プロセス数
```

このスクリプトは以下の 3 ステップを自動実行する：

1. generate_analytical.py で解析解を生成 (ref/*.semd)
2. semsws でシミュレーション実行
3. 各トレースの正規化 L2 誤差を算出・表示 (6 局 × 3 成分 = 18 トレース)

6.2.3 結果の可視化

```
python3 plot_comparison.py
```

3 局 × 3 成分の波形比較図が生成される。解析解（黒実線）、数値解（赤破線）、残差 ×10（青線）が重ねて表示される。

6.3 visco_elastic_3d_layered — 3D 粘弾性 2 層モデル

examples/visco_elastic_3d_layered/は、CUBIT で生成した外部メッシュを用いた 3 次元 2 層モデルの例題である。メッシュの評価・細分化のワークフローについては第 5 章の第 5.5 節で詳述した。

6.3.1 含まれる設定

ファイル	説明
config_elastic.yaml	弾性体（減衰なし）、外部メッシュ直接読み込み
config_elastic_sim.yaml	弾性体、細分化済み partitioned メッシュ
config_visco.yaml	粘弾性体（減衰あり）、外部メッシュ直接読み込み
config_visco_sim.yaml	粘弾性体、細分化済み partitioned メッシュ
mesh_layered_3d.jou	CUBIT ジャーナルファイル
materials_elastic.txt	弾性物性 ($Q=9999$ 、実質減衰なし)
materials_visco.txt	粘弾性物性 ($Q_\kappa = Q_\mu = 50-100$)

6.3.2 実行方法

```
cd examples/visco_elastic_3d_layered

# 1. CUBITでメッシュ生成
coreform_cubit -nographics -input mesh_layered_3d.jou

# 2. メッシュ細分化 (64プロセス)
mpirun -np 64 ../../build/src/refine_mesh \
  --config config_visco.yaml --output-dir mesh/

# 3. シミュレーション実行
mpirun -np 64 ../../build/src/semsws \
  --config config_visco_sim.yaml
```

または付属のスクリプトで一括実行：

```
bash run_visco.sh
```

弾性体と粘弾性体の波形を比較するスクリプトも用意されている：

```
python3 plot_comparison.py
```

6.4 fun

examples/fun/には、HOHQMesh ライブラリで生成した非構造メッシュを用いた例題がある。コーディングに疲れて適当に作ったので設定などに問題があるかもしれない。HOHQMesh はオープンソースの高次六面体/四面体メッシュ生成ツールであり、テキストベースの入力ファイルで複雑な形状のメッシュを作成できる。

これらの例題は CUBIT/GMSH 以外のメッシュソースを使用する実例として参考になる。

6.4.1 ICM — 2D 弾性波

`examples/fun/icm/`は「ICM」の文字形状メッシュ上での 2D 弾性波シミュレーションである。「i」の上部の点に震源を配置し、フォワードシミュレーションで波が広がる様子を計算した後、スナップショットを逆順に並べることで波が「i」の点に収束していくアニメーションを生成する。

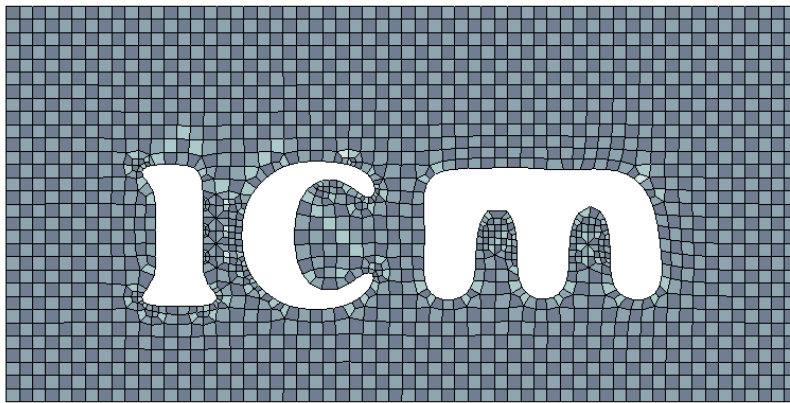


図 6.1 HOHQMESH で生成した「ICM」文字形状メッシュ。

```
cd examples/fun/icm
bash run.sh
```

ファイル	説明
<code>icm.control</code>	HOHQMESH の制御ファイル
<code>generate_mesh.py</code>	HOHQMESH の出力を GMSH 形式に変換
<code>config.yaml</code>	シミュレーション設定 (2D elastic, order 8)
<code>make_animation.py</code>	スナップショットからアニメーション生成

デモ動画 (逆再生アニメーション): https://youtu.be/Qts2n_2wtfI

6.4.2 JAMSTEC — 2D 音響波

`examples/fun/jamstec/`は「JAMSTEC」の文字形状メッシュ上での 2D 音響波シミュレーションである。ICM と同様に、「J」の上部の点に震源を配置し、逆再生で波が収束するアニメーション

を生成する。



図 6.2 HOHQMesh で生成した「JAMSTEC」文字形状メッシュ。

```
cd examples/fun/jamstec
bash run.sh
```

デモ動画（逆再生アニメーション）：https://youtu.be/_yijXLc3F6U

これらの例題では、HOHQMesh で生成したメッシュ（.msh）を `mesh.type: external` として読み込んでいる。同様の手順で任意の形状のメッシュを使用したシミュレーションが可能である。

第 7 章

パフォーマンス

本章では、Earth Simulator (ES, JAMSTEC) および LUMI スーパーコンピュータ (CSC, Finland) における SEMSWS の並列性能を示す。これらの結果は論文から引用したものである。

7.1 テスト環境

表 7.1 テストに使用したシステム

システム	パーティション	構成
ES (JAMSTEC)	CPU	AMD EPYC 7742 (128 コア/ノード)、720 ノード
ES (JAMSTEC)	GPU	NVIDIA A100 (8 GPU/ノード)、8 ノード
LUMI (CSC)	GPU	AMD MI250X (4 モジュール = 8 GCD/ノード)、2978 ノード

すべてのテストは 3 次元等方弾性波動方程式を 4 次多項式 (5 GLL 点) で 1000 タイムステップ計算する。メッシュは構造六面体メッシュ、分割は Cartesian 分割、GPU 実行では GPU-aware MPI を使用した。

7.2 Strong Scaling

Strong scaling テストでは問題サイズを 524 万要素 (3 億 3730 万メッシュノード) に固定し、プロセス数を増加させる。

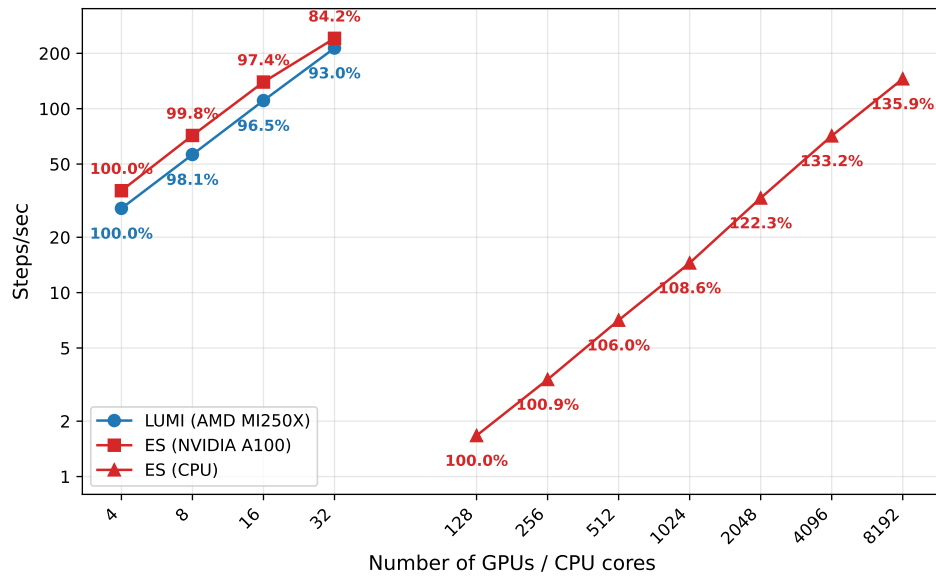


図 7.1 Strong scaling 結果。ES CPU ノード (AMD EPYC 7742)、ES GPU ノード (NVIDIA A100)、LUMI GPU ノード (AMD MI250X)。各シンボルの横の数値は並列効率 (%)。基準は CPU: 128 コア、GPU: 4 基。

ES CPU では 8192 コアまで super-linear scaling（並列効率 135.9%）が観測された。これはコアあたりの問題サイズ減少に伴うキャッシュ効率の向上によるものと考えられる。GPU では、ES で 84.2%、LUMI で 93.0% の並列効率を 32 GPU まで維持した。

7.3 Weak Scaling

Weak scaling テストでは CPU あたり 15,210 要素、GPU あたり 1,600,000 要素に固定し、プロセス数に比例して問題サイズを増加させる。

表 7.2 Weak scaling テストの問題サイズ

CPU (ES, 15,210 要素/コア)				GPU (ES & LUMI, 1,600,000 要素/GPU)			
nCPU	要素数	メッシュノード	DoFs	nGPU	要素数	メッシュノード	DoFs
128	1.95M	125.4M	376.2M	4	6.40M	412.7M	1.24B
512	7.79M	500.8M	1.50B	8	12.8M	825.1M	2.48B
2048	31.2M	2.00B	6.00B	16	25.6M	1.65B	4.95B
8192	124.6M	8.00B	24.0B	32	51.2M	3.30B	9.90B
				GPU (LUMI のみ)			
				320	512.0M	32.8B	98.4B
				640	1.02B	65.6B	196.8B
				1280	2.05B	131.2B	393.6B
				2560	4.10B	262.4B	787.2B
				5120	8.19B	524.8B	1.57T
				8192	13.1B	839.6B	2.52T

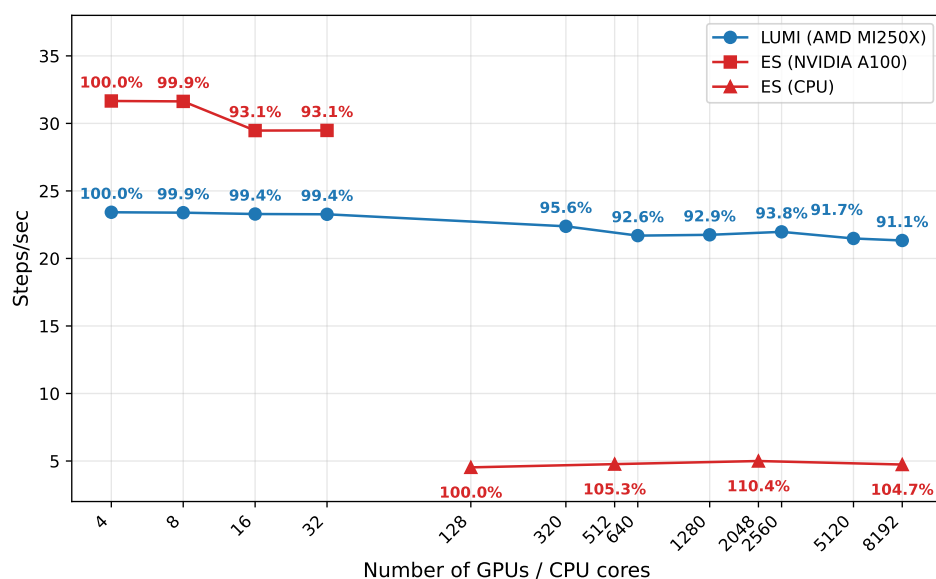


図 7.2 Weak scaling 結果。ES CPU ノード、ES GPU ノード、LUMI GPU ノード。基準は CPU: 128 コア、GPU: 4 基。CPU と GPU では 1 プロセスあたりの問題サイズが異なる点に注意。

CPU では 8192 コアまで並列効率 100% 以上を維持した。GPU 計算では、ES で 32 NVIDIA A100 まで 93% 以上、LUMI で 8192 AMD MI250X まで 91% 以上の並列効率を達成した。最大構成 (LUMI 8192 GPU) では 8396 億メッシュノード (2.52 兆 DoFs) の計算を実現した。

7.4 GPU vs CPU

ES での strong scaling 結果を用いて、1 GPU ノード (8 NVIDIA A100) と N CPU ノード (128 コア/ノード) の性能を比較する。

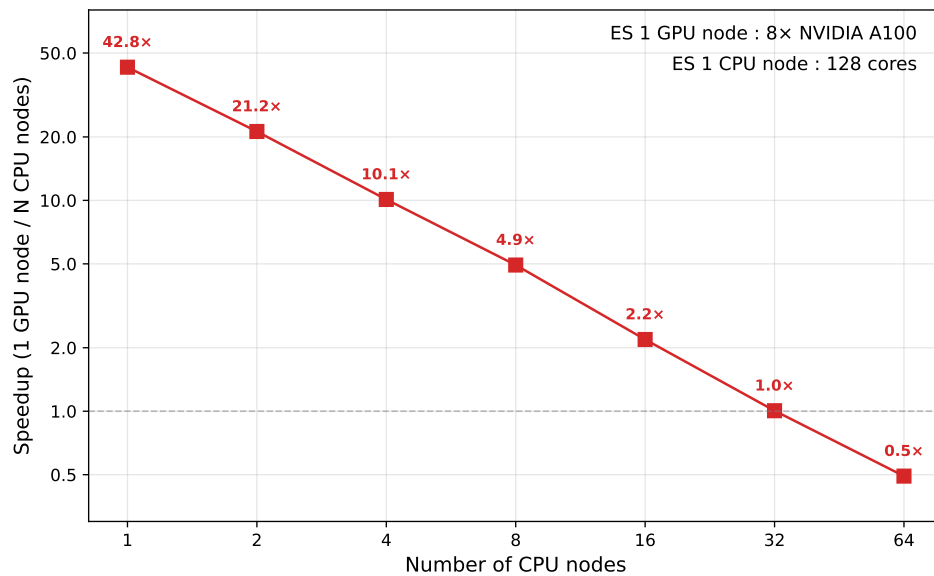


図 7.3 1 GPU ノード (8 NVIDIA A100) の N CPU ノード (128 コア/ノード) に対するスピードアップ。問題サイズは 524 万要素。

1 GPU ノードは 1 CPU ノードに対して 42.8 倍のスピードアップを達成した。1 GPU ノードの性能は 32 CPU ノード (4,096 コア) に相当する。

開発チーム

- Kota Mukumoto — Barcelona Center for Subsurface Imaging, Institut de Ciències del Mar (CSIC), Barcelona, Spain
- Yann Capdeville — Nantes Université, Univ Angers, Le Mans Université, CNRS, Laboratoire de Planétologie et Géosciences, Nantes, France
- Kazuya Shiraishi — Japan Agency for Marine-Earth Science and Technology (JAMSTEC), Japan
- Ryoichiro Agata — Japan Agency for Marine-Earth Science and Technology (JAMSTEC), Japan
- Gou Fujie — Japan Agency for Marine-Earth Science and Technology (JAMSTEC), Japan
- Thomas Bodin — Barcelona Center for Subsurface Imaging, Institut de Ciències del Mar (CSIC), Barcelona, Spain